

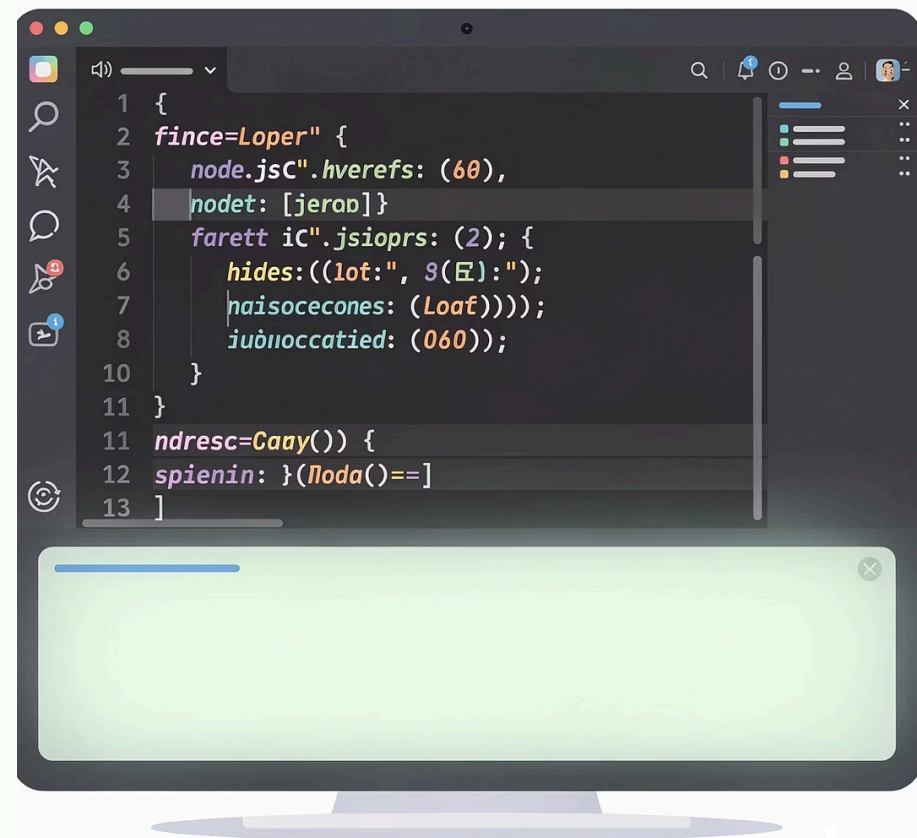
ОСНОВИ JavaScript + Telegraf

У цьому модулі ти напишеш справжнього Telegram-бота на Node.js – з нуля до робочого коду.

NODE.JS

TELEGRAF

JAVASCRIPT



Мінімальний робочий бот

```
const { Telegraf } = require('telegraf');  
const bot = new Telegraf('ТВИЙ_TOKEN');  
  
bot.start((ctx) => {  
  ctx.reply('Бот запущений!');  
});  
  
bot.launch();
```

require('telegraf')

Підключає
бібліотеку Telegraf

new Telegraf(...)

Створює екземпляр
бота з токеном

bot.start(...)

Обробляє команду
/start

bot.launch()

Запускає бота – без
цього нічого не
працює!

Дебаг через console.log

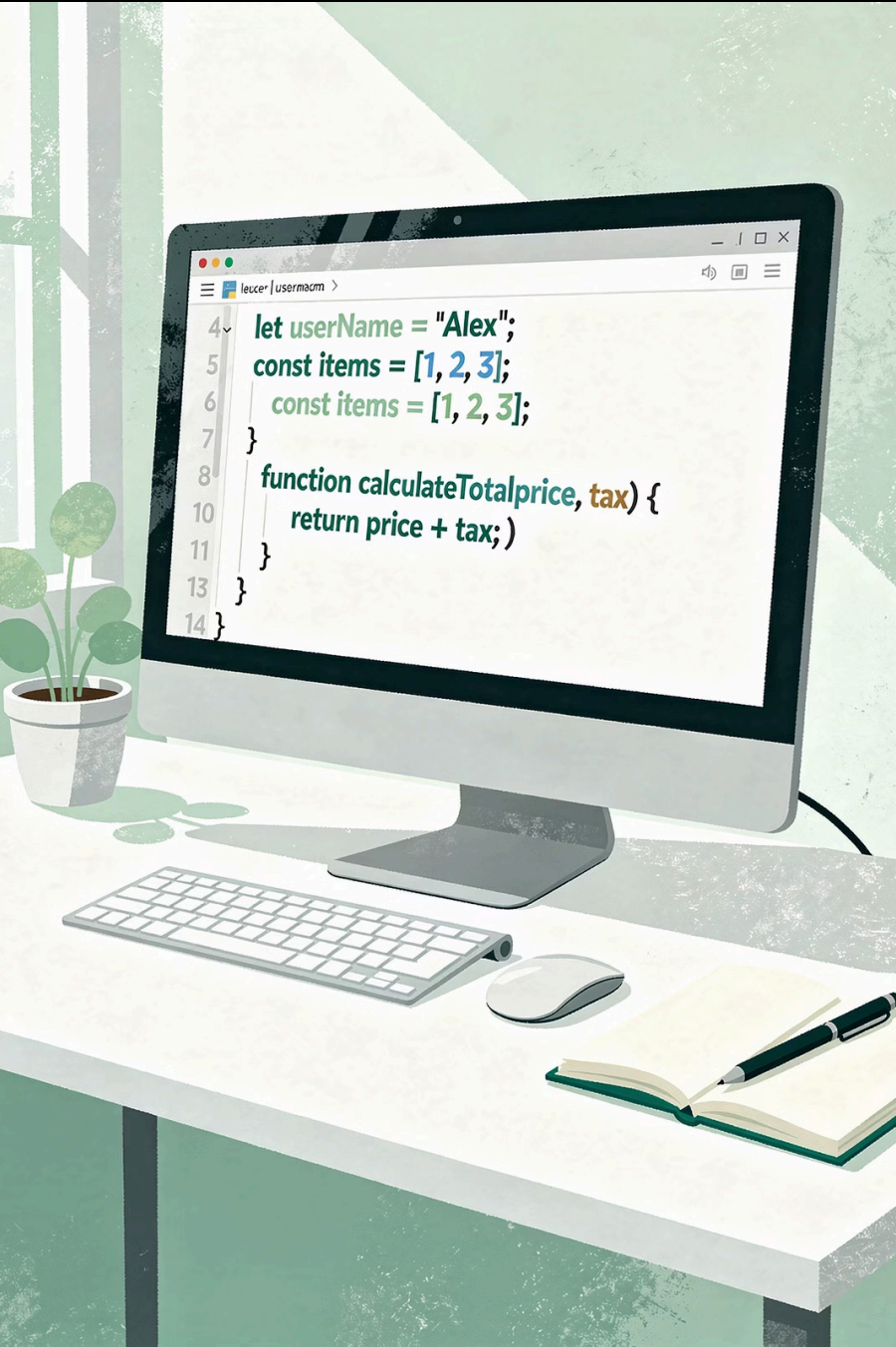
```
bot.start((ctx) => {  
  console.log(  
    "Отримано /start від:",  
    ctx.from.first_name  
  );  
  ctx.reply("Привіт!");  
});
```

Навіщо це потрібно?

- Бачиш, що відбувається всередині бота
- Перевіряєш, чи дішов запит
- Знаходиш помилки до того, як вони зламують усе



Термінал — твій найкращий друг при розробці.



Змінні у боті

```
bot.start((ctx) => {  
  const userName =  
    ctx.from.first_name;  
  const message = "Привіт,  
    " + userName + "!";  
  ctx.reply(message);  
});
```

const

Значення не змінюється
після присвоєння

let

Значення можна
змінювати пізніше

`ctx.from.first_name` – ім'я
користувача прямо з Telegram

Умови if/else

```
bot.on("text", (ctx) => {  
  const text = ctx.message.text;  
  
  if (text === "привіт") {  
    ctx.reply("І тобі привіт!");  
  } else {  
    ctx.reply("Я поки знаю тільки  
'привіт'");  
  }  
});
```

Як бот приймає рішення



Невідома команда

Надіслати підказку

Відповісти привіт

Надіслати «І тобі привіт!»

Перевірити рівність

`text === "привіт"?`

Отримати текст

Користувач надсилає повідомлення

Функції для структури коду

```
function getWelcomeMessage(name) {  
  return "Ласкаво просимо, " + name;  
}
```

```
bot.start((ctx) => {  
  const name = ctx.from.first_name;  
  ctx.reply(getWelcomeMessage(name));  
});
```

Навіщо виносити логіку у функції?

- Код стає читабельнішим
- Можна повторно використовувати в різних місцях
- Легше знайти і виправити помилку

📌 **Одна функція — одна задача.** Це основа чистого коду.

Масиви відповідей

```
const jokes = [
```

```
  "Чому програмісти не  
  виходять на вулицю? Бо  
  там немає Wi-Fi.",
```

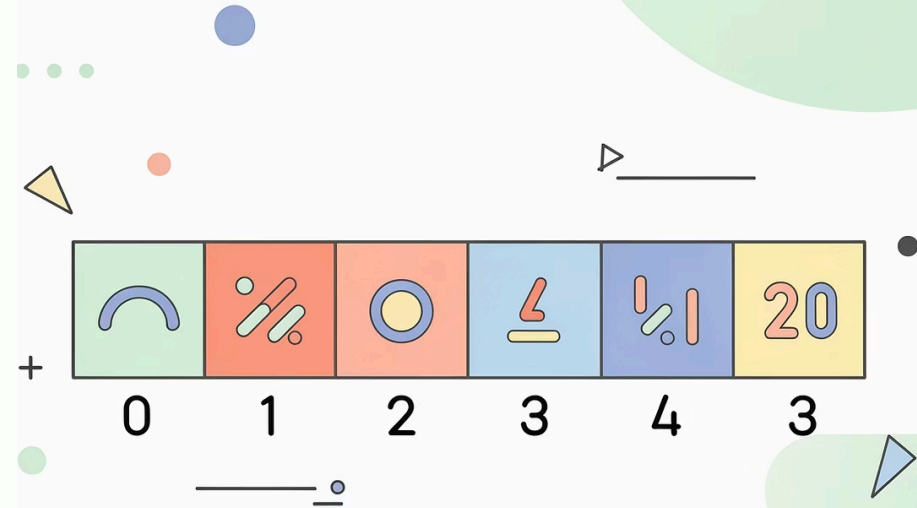
```
  "Скільки програмістів  
  треба, щоб замінити  
  лампочку? Жодного — це  
  апаратна проблема.",
```

```
  "Bug — це не помилка,  
  це недокументована  
  фіча."
```

```
];
```

Масив — це список значень

- Зберігає кілька варіантів в одній змінній
- Кожен елемент має індекс: `jokes[0]`, `jokes[1]`...
- Зручно для відповідей, меню, команд



Випадкова відповідь через **Math.random**

```
bot.command("joke", (ctx) => {  
  const randomIndex = Math.floor(  
    Math.random() * jokes.length  
  );  
  ctx.reply(jokes[randomIndex]);  
});
```

01

Math.random()

Генерує число від 0 до 1

02

× jokes.length

Масштабує до розміру масиву

03

Math.floor()

Округлює до цілого індексу

04

jokes[randomIndex]

Повертає випадковий жарт

Цикли

```
const commands = ["start", "help", "joke"];

for (let i = 0; i < commands.length; i++) {
  console.log(commands[i]);
}
```

```
// Виведе:
// start
// help
// joke
```

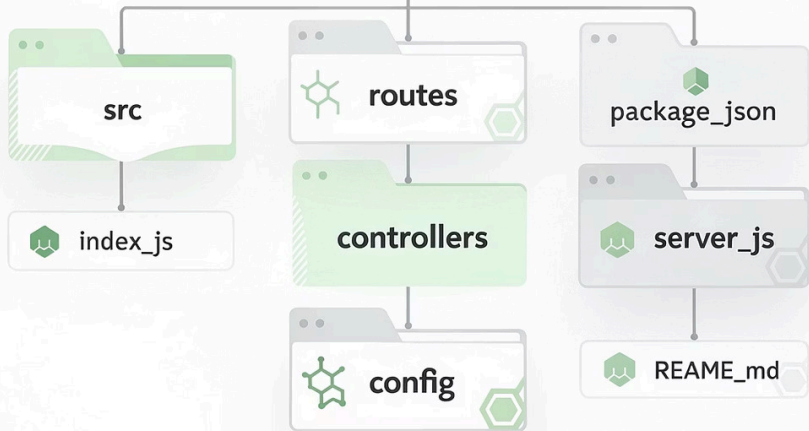
Де цикл знадобиться у боті?

- Автоматична генерація пунктів меню
- Перебір списку команд
- Обробка масиву повідомлень

📌 `i` – це лічильник. Починає з 0, бо масиви в JS теж з 0.

```
project" { node.jss.jog" {
{
  nppm insstall[
    "rofeect: dovel{305"}do.[];
    ioodrt_ic project_ssc(2020516°));
    isdeel_6d_1829998);
    ioddrs_dc contjejed oing I(Es";
    isdeet.8c_1834846));
    isdecl_6SC_52128S"))
  };
  "/s/conf{::/cennen"; parrpakag_ision"" {
    sspet.iss{)}";
    /fnponc: /odre(2608pBZ"))[]";
    ""Teore:(nodje.jsn")56"), "ndex_js" import"js,server"}
  }
}
```

node-project



Модулі в Node.js

messages.js

```
const jokes = [
  "Жарт 1",
  "Жарт 2"
];

module.exports = jokes;
```

bot.js

```
const jokes =
require("../messages");
// jokes вже доступні тут!
```

Навіщо ділити код на файли?

Чистота

Кожен файл – своя зона відповідальності

Масштаб

Великий бот = багато файлів

Повторне використання

Один модуль – для всього проєкту

Що таке модуль?

Модуль — це окремий файл із кодом

У Node.js кожен файл .js автоматично є модулем. Модулі допомагають розділяти програму на частини, щоб код був зрозумілішим і зручнішим у підтримці.

Без модулів

Весь код в одному файлі – важко читати, важко підтримувати

З модулями

Кожна частина програми – в окремому файлі зі своєю відповідальністю

Типова структура проєкту бота

```
my-bot/  
  bot.js      - головний файл  
  messages.js - тексти та масиви  
  commands.js - обробники команд  
  config.js   - налаштування
```

📄 **Кожен .js файл у Node.js — це вже модуль. Нічого додатково встановлювати не потрібно.**

Експорт коду з модуля

Файл `messages.js`

```
const jokes = [  
  "Жарт 1",  
  "Жарт 2",  
  "Жарт 3"  
];  
  
module.exports = jokes;
```

Ми робимо масив `jokes` доступним для інших файлів через `module.exports`.

Як працює `module.exports`?

01

Створюємо дані

Оголошуємо змінну, масив або функцію у файлі

02

Експортуємо

`module.exports` = назва – робимо доступним ззовні

03

Готово до використання

Інші файли можуть підключити цей модуль



Можна експортувати будь-що: масив, об'єкт, функцію або навіть рядок

Імпорт модуля через `require`

Файл `bot.js`

```
const jokes = require("./messages");  
  
console.log(jokes[0]); // "Жарт 1"
```

"./messages" означає файл `messages.js` у цій же папці. Крапка і слеш – це відносний шлях.

Що означає `"./messages"`?



. (крапка)

Поточна папка, де знаходиться файл



/ (слеш)

Розділювач шляху до файлу



messages

Назва файлу без розширення `.js`



Для вбудованих модулів Node.js (наприклад `fs`, `path`) крапку не пишуть: `require("fs")`

Модулі у Telegram-боті — приклад

messages.js

```
const jokes = ["Жарт 1", "Жарт 2"];

module.exports = jokes;
```

bot.js

```
const jokes = require("./messages");

bot.command("joke", (ctx) => {
  ctx.reply(jokes[0]);
});
```

Навіщо ділити код на модулі?

- Розділяти великий код на файли – легше читати
- Перевикористовувати функції в різних місцях
- Тримати логіку бота структурованою
- Легше знаходити і виправляти помилки

Типовий розподіл у боті:

1

bot.js

2

commands.js

3

messages.js

4

config.js



Чим більший бот — тим важливіше правильно розділяти код на модулі

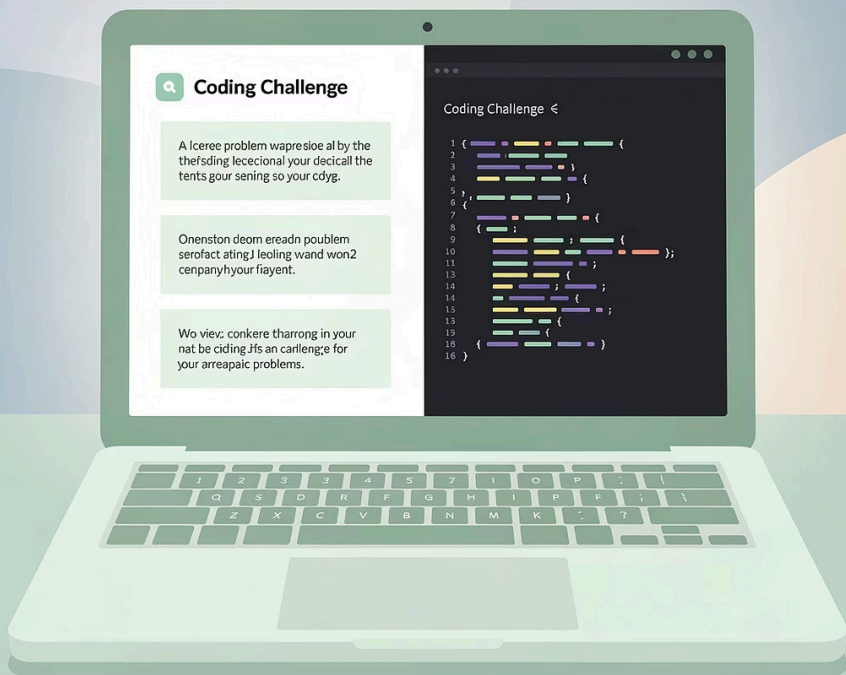
Обробка кількох команд

```
bot.command("start", (ctx) => {  
  ctx.reply("Я працюю!");  
});  
  
bot.command("help", (ctx) => {  
  ctx.reply(  
    "Доступні команди:\n/start\n/help\n/joke"  
  );  
});  
  
bot.command("joke", (ctx) => {  
  const i = Math.floor(Math.random() * jokes.length);  
  ctx.reply(jokes[i]);  
});
```

Масштабування бота

- Кожна команда – окремий обробник
- Додати нову команду – додати один блок коду
- Структура залишається чистою

📌 Чим більше команд – тим важливіше ділити код на модулі.



МІНІ-ПРОЄКТ 1

Бот із трьома командами

1

Три команди

Реалізує `/start`, `/help`, `/joke`

2

Масив відповідей

Жарти або факти –
зберігати у масиві

3

Запусти і протестуй

Надішли кожну команду у Telegram

Бот із випадковими відповідями

Завдання

```
// Масив із 5+ відповідей
const answers = [ ... ];

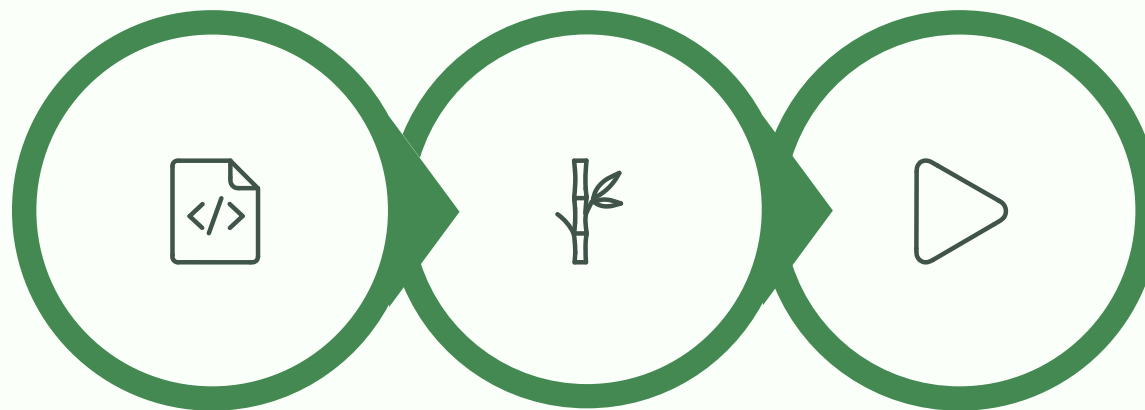
// При команді /random —
// видати випадкову відповідь
bot.command("random", (ctx) => {
  // твій код тут
});
```

Що відпрацьовуємо

- `Math.random() + Math.floor()`
- Робота з індексами масиву
- Обробка команди через `bot.command`

📌 **Бонус:** зроби масив із 10+ різних відповідей для більшої різноманітності.

Код у модулях



messages.js

bot.js

Запуск

Розділи бота на два файли: `messages.js` зберігає дані, `bot.js` містить логіку. Переконайся, що все працює після рефакторингу.

Дебаг і типові помилки

✗ Неправильний токен

Перевір токен від @BotFather – жодного зайвого символу

✗ `bot.launch()` не викликаний

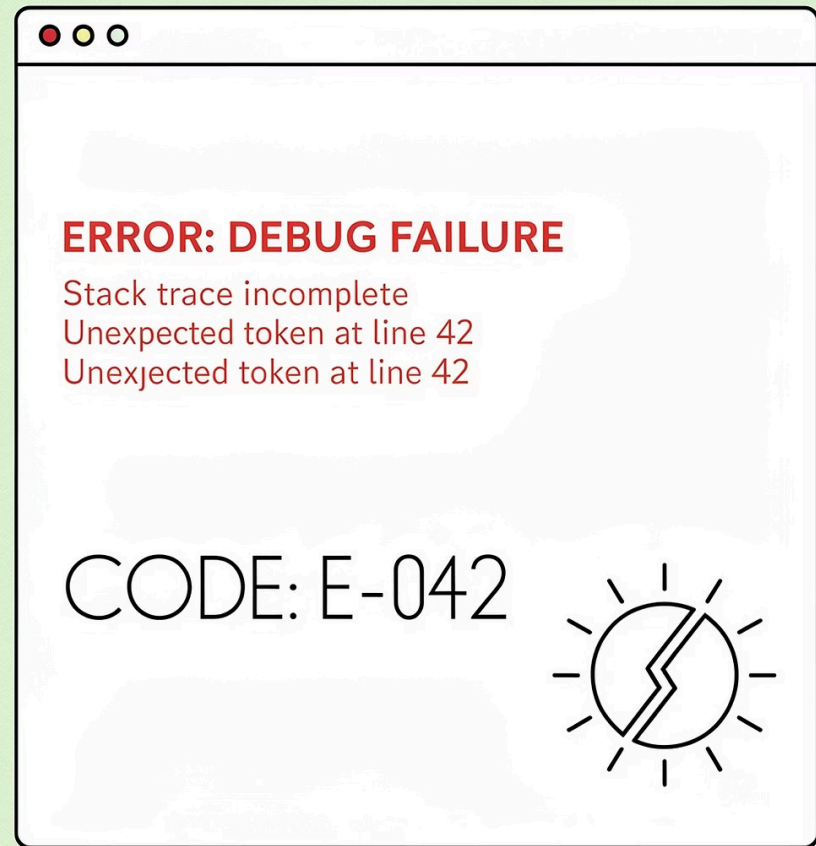
Без цього рядка бот ніколи не запуститься

✗ Синтаксична помилка

Пропущена дужка або лапка – термінал покаже рядок

✗ Бот не запущений

Запусти `node bot.js` у терміналі перед тестуванням



Підсумок модуля



Змінні

const і let у реальному боті



Функції

Чистий і структурований код



Цикли

Перебір команд і даних



Умови

if/else для логіки відповідей



Масиви

Кілька відповідей в одній змінній



Модулі

require і module.exports

📌 Ти вже пишеш справжніх Telegram-ботів на Node.js. 🚀

Кнопки, повідомлення, сесії та логіка Telegram-ботів

У цьому модулі бот стане справжньо інтерактивним: з кнопками, медіа, іграми та збереженням стану користувача.



Кнопки

Reply та Inline клавіатури



Медіа

Фото, відео, файли



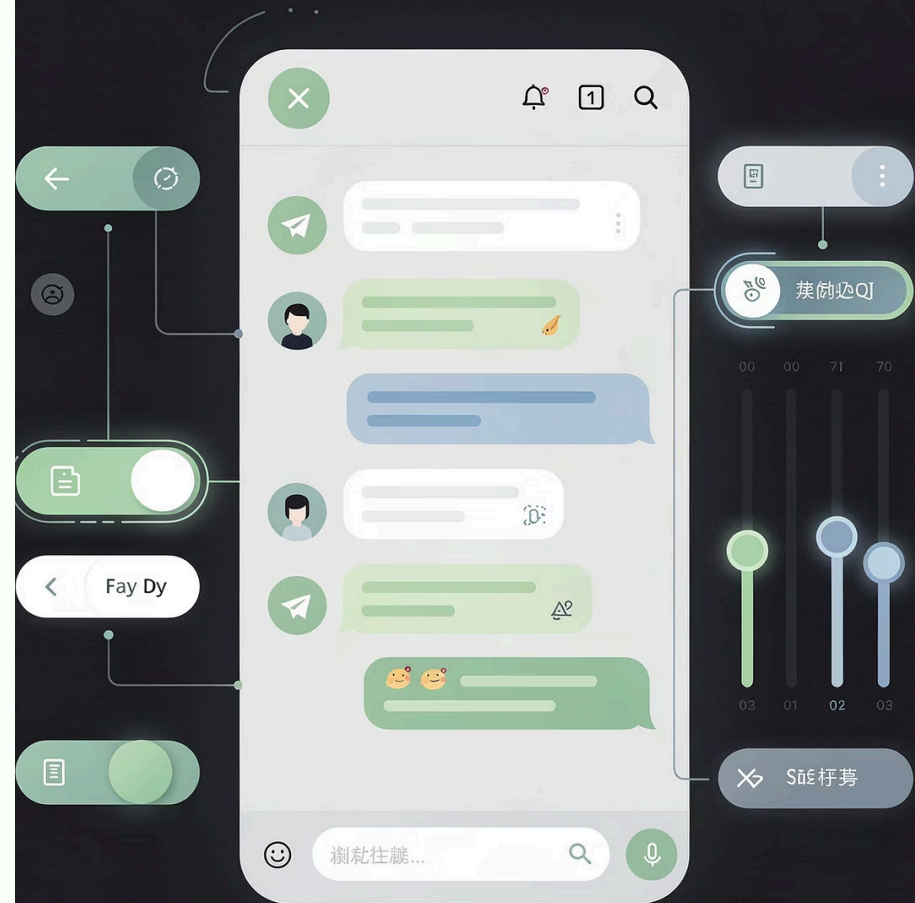
Сесії

Збереження стану користувача



Ігри

Вікторини та міні-ігри



Reply клавiатура (звичайні кнопки)

```
bot.start((ctx) => {  
  ctx.reply("Обери дію", {  
    reply_markup: {  
      keyboard: [  
        ["Інфо", "Допомога"],  
        ["Жарт"]  
      ],  
      resize_keyboard: true  
    }  
  });  
});
```

Що таке Reply клавiатура?

- Замінює стандартну клавiатуру користувача
- Кнопки відображаються внизу екрана
- Натискання надсилає текст кнопки як повідомлення
- `resize_keyboard: true` – підлаштовує розмір під кнопки

📌 **Reply клавiатура — найпростіший спосіб додати кнопки до бота**

Inline кнопки

```
const { Markup } = require("telegraf");

bot.start((ctx) => {
  ctx.reply(
    "Меню",
    Markup.inlineKeyboard([
      Markup.button.callback("Жарт", "joke"),
      Markup.button.callback("Факт", "fact")
    ])
  );
});
```

Reply vs Inline — різниця

Reply клавіатура

Замінює клавіатуру телефону. Надсилає текст як повідомлення.

Inline кнопки

З'являються прямо під повідомленням. Надсилають callback_data, не текст.

Обробка натискання кнопки

```
bot.action("joke", (ctx) => {  
  ctx.reply("Ось випадковий жарт");  
});
```

Що таке `callback_data`?

- `bot.action()` слухає натискання inline кнопок
- "joke" – це `callback_data`, яку ми задали у `Markup.button.callback`
- `ctx.reply()` надсилає відповідь користувачу
- Можна обробляти скільки завгодно різних кнопок

📄 `callback_data` — це "секретний код" кнопки, який бот отримує при натисканні

ПРАКТИКА

Меню навігації

Завдання: створити меню з кнопками



Nacheure



Stakenurse



Choring Beat



Utestany



/start

Бот вітає користувача і показує меню



Інфо

Бот надсилає інформацію про себе



Допомога

Бот показує список доступних команд



Кожна кнопка повинна викликати окрему відповідь бота через `bot.action()`

Надсилання картинок

```
bot.command("photo", (ctx) => {  
  ctx.replyWithPhoto("https://picsum.photos/300");  
});
```

Як працює `replyWithPhoto`?

- `ctx.replyWithPhoto()` надсилає зображення у чат
- Можна передати URL або `file_id` з Telegram
- Можна додати підпис через параметр `caption`
- Підтримує JPEG, PNG, GIF та інші формати

📄 `picsum.photos` – зручний сервіс для тестових зображень

Надсилання відео

```
bot.command("video", (ctx) => {  
  ctx.replyWithVideo(  
    "https://sample-  
videos.com/video123/mp4/240/big_buck_bunny_240p  
_1mb.mp4"  
  );  
});
```

Що потрібно знати про відео?

- `ctx.replyWithVideo()` надсилає відеофайл у чат
- Telegram підтримує MP4 формат
- Можна передати URL або `file_id`
- Максимальний розмір файлу – 50 МБ через URL
- Можна додати підпис через `caption`

❏ Для великих відео краще використовувати `file_id` після першого завантаження

Надсилання файлів

```
bot.command("file", (ctx) => {  
  ctx.replyWithDocument(  
    "https://example.com/file.pdf"  
  );  
});
```

Боти можуть надсилати будь-які файли

- `ctx.replyWithDocument()` надсилає файл будь-якого типу
- PDF, DOCX, ZIP, MP3 – все підтримується
- Можна передати URL або `file_id`
- Файл відображається як вкладення у чаті



PDF



ZIP



MP3



XLSX

Читання повідомлень користувача

```
bot.on("text", (ctx) => {  
  const text = ctx.message.text;  
  ctx.reply("Ти написав: " + text);  
});
```

Що таке `ctx.message.text`?

- `bot.on("text")` спрацьовує на будь-яке текстове повідомлення
- `ctx.message.text` – це текст, який написав користувач
- `ctx.from.first_name` – ім'я користувача
- `ctx.from.id` – унікальний ID користувача
- `ctx.chat.id` – ID чату

📌 **ctx** – це контекст повідомлення. Він містить всю інформацію про відправника і чат.

Бот-чат

Завдання: створити бота, який відповідає на різні слова

привіт

Бот відповідає: "І тобі привіт!"



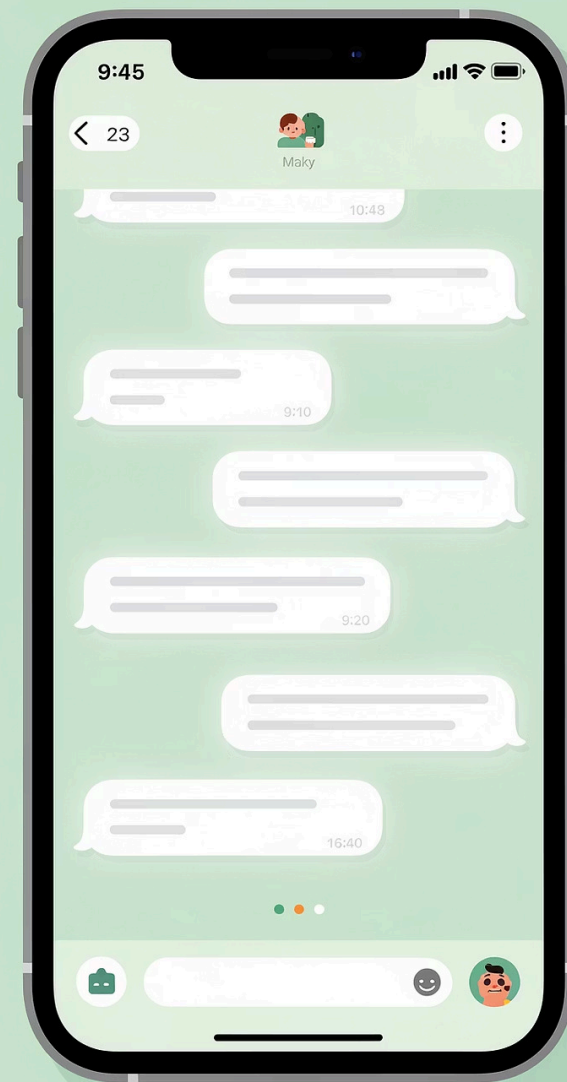
жарт

Бот надсилає випадковий жарт з масиву

help

Бот показує список доступних команд

📌 Використати if/else або switch для обробки різних слів



Навіщо потрібні сесії?

Проблема

Бот не пам'ятає попередніх повідомлень.
Кожне повідомлення – це окремий запит.

Без сесій

Бот не знає, що ти вже відповів на перше питання анкети

З сесіями

Бот пам'ятає твої крок, ім'я, вік та інші дані

Де потрібні сесії?

-  Анкети та опитування
-  Ігри (рахунок, поточний рівень)
-  Кошик у магазині-боті
-  Авторизація та профіль користувача



Сесія – це об'єкт, де ми зберігаємо дані конкретного користувача під час роботи бота

Проста сесія на об'єкті

```
const sessions = {};  
  
bot.on("text", (ctx) => {  
  const userId = ctx.from.id;  
  
  if (!sessions[userId]) {  
    sessions[userId] = { step: 1 };  
  }  
  
  ctx.reply("Твій крок: " + sessions[userId].step);  
});
```

Як це працює?

01

Отримуємо `userId`

`ctx.from.id` – унікальний ID кожного користувача

02

Перевіряємо сесію

Якщо сесії немає – створюємо нову

03

Зберігаємо стан

`sessions[userId].step` зберігає поточний крок

04

Відповідаємо

Бот знає, на якому кроці знаходиться користувач

ПРАКТИКА

Бот-анкета

Логіка: бот задає питання по черзі

1

Крок 1

Бот запитує: "Як тебе звати?"

2

Крок 2

Бот запитує: "Скільки тобі років?"

3

Крок 3

Бот запитує: "З якого ти міста?"



Після всіх відповідей бот показує: "Ім'я: ..., Вік: ..., Місто: ..."



Бот-анкета: повний код

bot.js

```
const { Telegraf } = require("telegraf");
const bot = new Telegraf("ТБІЙ_ТОКЕН");

const sessions = {};

bot.start((ctx) => {
  const id = ctx.from.id;
  sessions[id] = { step: 1 };
  ctx.reply("Як тебе звати?");
});

bot.on("text", (ctx) => {
  const id = ctx.from.id;
  const text = ctx.message.text;

  if (!sessions[id]) return;

  if (sessions[id].step === 1) {
    sessions[id].name = text;
    sessions[id].step = 2;
    ctx.reply("Скільки тобі років?");
  } else if (sessions[id].step === 2) {
    sessions[id].age = text;
    sessions[id].step = 3;
    ctx.reply("З якого ти міста?");
  } else if (sessions[id].step === 3) {
    sessions[id].city = text;
    const s = sessions[id];
    ctx.reply(
      "Ім'я: " + s.name +
      "\nВік: " + s.age +
      "\nМісто: " + s.city
    );
    delete sessions[id];
  }
});

bot.launch();
```

Як це працює?

01

bot.start()

Ініціалізує сесію і запитує ім'я

02

step === 1

Зберігає ім'я, переходить до кроку 2

03

step === 2

Зберігає вік, переходить до кроку 3

04

step === 3

Зберігає місто і виводить підсумок

Бот-довідник

Завдання: зробити бота з меню та кнопками



/start

Показує привітання та inline-меню з кнопками



/help

Список всіх доступних команд бота



/info

Інформація про бота та його можливості



Користувач обирає розділ через кнопки, бот відповідає відповідним контентом



Random у JavaScript

```
const randomNumber = Math.floor(  
  Math.random() * 10  
) + 1;  
  
console.log(randomNumber); // від 1 до 10
```

Як це працює?

01

Math.random()

Генерує випадкове число від 0 до 1 (наприклад, 0.742)

02

*** 10**

Множимо на 10, щоб отримати число від 0 до 9.99

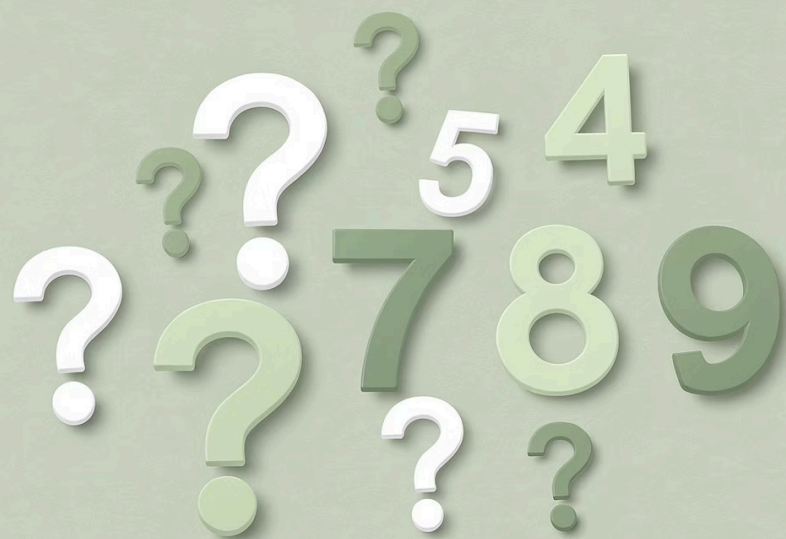
03

Math.floor() + 1

Округлюємо вниз і додаємо 1 → отримуємо від 1 до 10



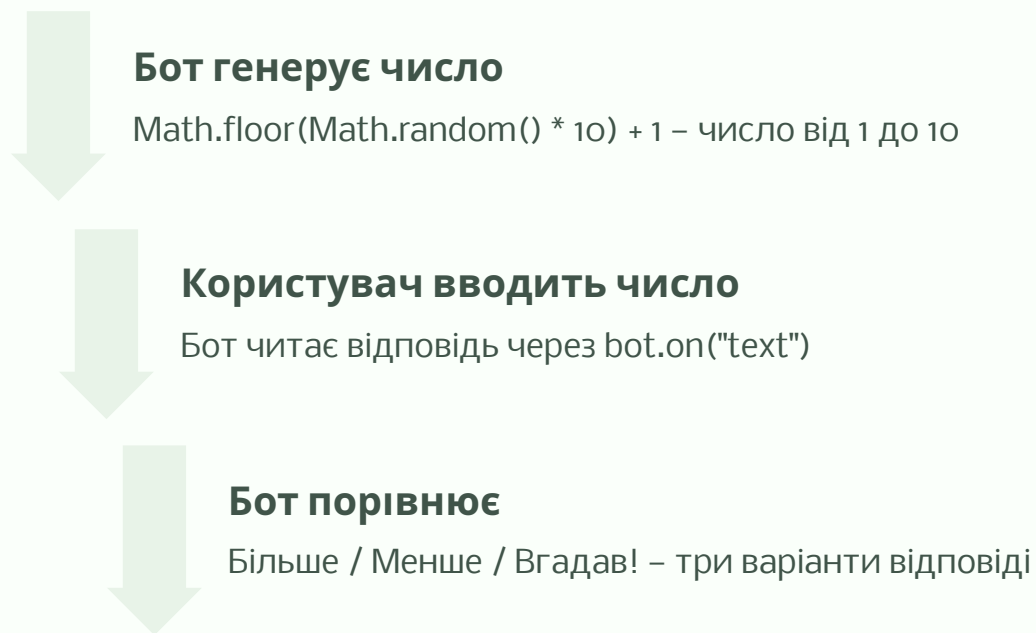
Math.random() * array.length – класичний спосіб отримати випадковий елемент масиву



ПРАКТИКА

Гра "Вгадай число"

Логіка гри:



- ❑ Зберігати загадане число у сесії: `sessions[userId].secret = randomNumber`

Кнопки-варіанти (quiz style)

```
bot.command("quiz", (ctx) => {  
  ctx.reply(  
    "Скільки буде 2 + 2?",  
    Markup.inlineKeyboard([  
      Markup.button.callback("3", "q1"),  
      Markup.button.callback("4", "q2"),  
      Markup.button.callback("5", "q3")  
    ])  
  );  
});
```

Як побудувати вікторину?

- Питання надсилається як текст повідомлення
- Варіанти відповідей – inline кнопки
- Кожна кнопка має свій callback_data (q1, q2, q3)
- bot.action() обробляє натискання кожної кнопки

❏ Можна додати кілька питань, зберігаючи рахунок у сесії

Обробка відповіді у вікторині

```
bot.action("q2", (ctx) => {  
  ctx.reply("Правильно!");  
});  
  
bot.action("q1", (ctx) => {  
  ctx.reply("Неправильно");  
});
```

Що відбувається при натисканні?

Правильна відповідь (q2)

Бот надсилає повідомлення про успіх і може перейти до наступного питання

Неправильна відповідь (q1, q3)

Бот повідомляє про помилку і може дати підказку або повторити питання

- ❑ Можна додати лічильник правильних відповідей у сесії користувача

Ідеї для розширення бота



Бот-вікторина

Питання з варіантами відповідей та підрахунком балів



Бот-гра

Вгадай число, хрестики-нулики, текстові пригоди



Бот-довідник

Структуроване меню з корисною інформацією



Бот-меми

Надсилає випадкові меми або картинки за командою



Бот-рандомні факти

Цікаві факти з масиву або зовнішнього API



Підсумок модуля

Після цього модуля ти вмієш будувати справжньо інтерактивних Telegram-ботів!



Кнопки

Reply та Inline клавіатури



Читання повідомлень

`bot.on("text")` та `ctx.message.text`



Медіа

Фото, відео та файли



Інтерактивні меню

Навігація через inline кнопки



Random

`Math.random()` для ігор та жартів



Стан користувача

Сесії для збереження даних



Міні-ігри

Вікторини та гра "Вгадай число"



Ти готовий створювати повноцінних інтерактивних Telegram-ботів! 🚀

Підключення API

Що таке API?

API (Application Programming Interface) дозволяє отримувати дані з інтернету – погоду, картинки, цитати тощо.

Бот може використовувати ці дані і надсилати їх користувачу.



Погода

Отримуй актуальну температуру та прогноз



Картинки

Надсилає фото котиків, мемів та іншого



Цитати

Випадкові мотиваційні або смішні цитати

axios для запитів

Встановлення

```
npm install axios
```

Підключення

```
const axios = require("axios");
```

Навіщо axios?

01

Надсилає HTTP-запит до API

02

Отримує відповідь у форматі JSON

03

Ти використовуєш дані у боті

Приклад API (котики)

```
const axios = require("axios");

axios.get("https://api.thecatapi.com/v1/images/search")
  .then((response) => {
    console.log(response.data);
  });
```

Як це працює?

01

axios.get()

Надсилає GET-запит до API котиків

02

response.data

Містить масив із даними про зображення

03

console.log()

Виводить URL картинки у термінал

Надсилання котика в боті

```
const axios = require("axios");

bot.command("cat", async (ctx) => {
  const response = await axios.get(
    "https://api.thecatapi.com/v1/images/search"
  );
  const imageUrl = response.data[0].url;
  ctx.replyWithPhoto(imageUrl);
});
```

Ключові моменти

- `async/await` – сучасний спосіб роботи з асинхронним кодом
- `response.data[0].url` – перший елемент масиву, поле `url`
- `ctx.replyWithPhoto()` – надсилає зображення у чат
- Кожен виклик `/cat` – нова випадкова картинка

API погоди

```
bot.command("weather", async (ctx) => {  
  const response = await axios.get(  
    "https://api.open-meteo.com/v1/forecast" +  
    "?latitude=50&longitude=30" +  
    "&current_weather=true"  
  );  
  const temp =  
    response.data.current_weather.temperature;  
  ctx.reply("Температура: " + temp + "°C");  
});
```

Що тут відбувається?

01

URL з параметрами

latitude і longitude – координати Києва

02

current_weather

Поточна погода у відповіді API

03

temperature

Числове значення температури у °C

Практика: API у боті

Завдання

Зроби команду /cat або /weather, яка отримує дані з API і надсилає користувачу.

```
// Варіант 1: котик
bot.command("cat", async (ctx) => {
  // отримай URL з thecatapi.com
  // надішли фото через replyWithPhoto
});
```

```
// Варіант 2: погода
bot.command("weather", async (ctx) => {
  // отримай температуру з open-meteo.com
  // надішли текст через ctx.reply
});
```

Підказки

- Не забудь `npm install axios` перед початком
- Використовуй `async/await` для запитів
- Перевір `response.data` у `console.log` спочатку

Збереження даних

Навіщо зберігати дані?

Бот може зберігати інформацію, щоб пам'ятати користувачів або результати між сесіями.



Користувачі

Пам'ятати ім'я, налаштування, прогрес



Статистика

Рахувати запуски, повідомлення, дії



Результати

Зберігати рахунок у грі або відповіді



JSON файл як база

Файл data.json

```
{  
  "visits": 0,  
  "users": {}  
}
```

Це простий спосіб зберігати дані між запусками бота. JSON – текстовий формат, зрозумілий і людині, і JavaScript.

Переваги JSON-файлу



Простота

Не потрібна база даних – лише файл



Читабельність

Можна відкрити і переглянути вручну



Швидкий старт

Ідеально для навчальних проєктів

Читання JSON

```
const fs = require("fs");

const data = JSON.parse(
  fs.readFileSync("data.json")
);

console.log(data.visits); // 0
console.log(data.users); // {}
```

Що тут відбувається?

01

fs.readFileSync()

Читає вміст файлу як текстовий рядок

02

JSON.parse()

Перетворює рядок у JavaScript-об'єкт

03

data.visits

Тепер можна звертатись до полів об'єкта

Запис у JSON

```
const fs = require("fs");

// Змінюємо дані
data.visits = 5;

// Записуємо назад у файл
fs.writeFileSync(
  "data.json",
  JSON.stringify(data, null, 2)
);
```

Параметри `JSON.stringify()`

DATA

Об'єкт, який потрібно зберегти

NULL

Замінник (не використовується)

2

Відступ у 2 пробіли для читабельності

Практика: лічильник запусків

Завдання

```
bot.start((ctx) => {  
  data.visits++;  
  fs.writeFileSync(  
    "data.json",  
    JSON.stringify(data)  
  );  
  ctx.reply("Запусків: " + data.visits);  
});
```

Як це зробити?

01

Створи файл `data.json` з `{ "visits": 0 }`

02

Зчитай дані через `fs.readFileSync` + `JSON.parse`

03

У `bot.start` збільшуй `data.visits++`

04

Записуй оновлені дані через `fs.writeFileSync`

Практика: збереження повідомлень

Завдання

```
bot.on("text", (ctx) => {  
  data.users[ctx.from.id] =  
    ctx.message.text;  
  fs.writeFileSync(  
    "data.json",  
    JSON.stringify(data)  
  );  
  ctx.reply("Збережено");  
});
```

Що тут відбувається?

- `ctx.from.id` – унікальний ID користувача в Telegram
- `data.users[id] = text` – зберігаємо останнє повідомлення
- Кожен користувач має свій запис у об'єкті `users`
- Після перезапуску бота дані зберезуться у файлі



МІНІ-ПРОЄКТ

Міні-проєкт: інтерактивна гра

Ідея гри

Бот загадує випадкове число від 1 до 10. Користувач вводить своє число. Бот відповідає «більше» або «менше» поки не вгадає.

Логіка

- Генерує число через `Math.random()`
- Зчитує відповідь через `bot.on text`
- Порівнює через `if/else` і відповідає

Що потрібно реалізувати?



Загадати число

`Math.random()` при старті гри



Читати відповідь

`bot.on text + parseInt()`



Порівняти

`if/else` для більше, менше, вгадав

Інтерактивна гра: повний код

```
const { Telegraf } = require("telegraf");
const bot = new Telegraf("ТБІЙ_ТОКЕН");

const sessions = {};

bot.command("start", (ctx) => {
  const userId = ctx.from.id;
  const secret = Math.floor(Math.random() * 10) + 1;
  sessions[userId] = { secret, attempts: 0 };
  ctx.reply("Я загадав число від 1 до 10. Вгадай!");
});
```

```
bot.on("text", (ctx) => {
  const userId = ctx.from.id;
  const session = sessions[userId];
  if (!session) {
    ctx.reply("Напиши /start щоб почати гру.");
    return;
  }
  const guess = parseInt(ctx.message.text);
  session.attempts++;
  if (isNaN(guess)) {
    ctx.reply("Введи число від 1 до 10.");
  } else if (guess < session.secret) {
    ctx.reply("Більше! Спробуй ще.");
  } else if (guess > session.secret) {
    ctx.reply("Менше! Спробуй ще.");
  } else {
    ctx.reply("Вгадав за " + session.attempts + " спроб!");
    delete sessions[userId];
  }
});

bot.launch();
```

Як це працює?



Старт гри

Math.random() генерує секретне число, зберігається в sessions



Введення числа

bot.on text зчитує відповідь, parseInt() перетворює на число



Порівняння

if/else повертає «більше», «менше» або «вгадав»



Кінець гри

delete sessions[userId] очищає сесію після перемоги

Міні-проєкт: бот-асистент

Команди бота



/cat

Надсилає випадкову картинку котика з API



/weather

Показує поточну температуру в Києві



/help

Виводить список доступних команд

Структура проєкту

- bot.js – головний файл бота
- data.json – збереження даних
- npm install telegraf axios
- Кожна команда – окремих bot.command()



Бот-асистент: повний код

```
const { Telegraf } = require("telegraf");
const axios = require("axios");
const fs = require("fs");

const bot = new Telegraf("ТБІЙ_ТОКЕН");

const data = JSON.parse(fs.readFileSync("data.json"));

bot.command("help", (ctx) => {
  ctx.reply(
    "Доступні команди:\n" +
    "/cat - котик\n" +
    "/weather - погода\n" +
    "/help - допомога"
  );
});

bot.command("cat", async (ctx) => {
  const res = await axios.get(
    "https://api.thecatapi.com/v1/images/search"
  );
  ctx.replyWithPhoto(res.data[0].url);
});
```

```
bot.command("weather", async (ctx) => {
  const res = await axios.get(
    "https://api.open-meteo.com/v1/forecast" +
    "?latitude=50&longitude=30&_weather=true"
  );
  const temp = res.data.current_weather.temperature;
  ctx.reply("Зараз у Києві: " + temp + " C");
});

bot.command("start", (ctx) => {
  data.visits = (data.visits || 0) + 1;
  fs.writeFileSync("data.json", JSON.stringify(data));
  ctx.reply(
    "Привіт! Це бот-асистент. Напиши /help."
  );
});

bot.launch();
```

Як це працює?



Підключення

telegraf, axios та fs підключаються на початку



Збереження

data.json зберігає лічильник запусків між перезапусками



API запити

axios отримує котика і погоду з зовнішніх API



Команди

/help, /cat, /weather – кожна як окремих bot.command()

Підсумок

Бот тепер вміє робити набагато більше – від запитів до API до збереження даних!



Робота з API

Отримувати дані з інтернету через axios



Надсилати дані

Фото, погода та інша інформація з мережі



Зберігати інформацію

JSON-файл як проста база даних



Проста логіка

if/else, цикли, функції у реальному боті



Міні-ігри

Вгадай число та інші інтерактивні механіки



Ти готовий створювати повноцінних Telegram-ботів з реальними даними! 🚀

Що таке вайб кодінг?

Вайб кодінг — це коли ти описуєш ідею словами, а штучний інтелект пише код замість тебе. Ти — режисер, AI — виконавець.



Пишеш словами

Замість того щоб знати синтаксис – просто описуєш що хочеш: «зроби бота який надсилає котика».



AI генерує код

Інструмент миттєво пише готовий код. Ти перевіряєш, запускаєш, просиш виправити якщо щось не так.



Ти вчишся швидше

Бачиш готовий код і розумієш як він працює. Вайб кодінг – не замість навчання, а разом з ним.



ВАЙБ КОДІНГ

Cursor — IDE з вбудованим AI

Cursor – це редактор коду як VS Code, але з потужним AI всередині. Можна просити його писати, пояснювати і виправляти код прямо в редакторі.

Як користуватись?



Встанови Cursor

Завантаж з cursor.com. Безкоштовний план є. Виглядає як звичайний VS Code – одразу знайомо.



Ctrl+K — редагуй код

Виділи код і натисни Ctrl+K. Напиши що змінити – AI переписує прямо на місці.



Ctrl+L — чат з кодом

Відкриває бічну панель. Питає про свій код: «поясни цей рядок», «додай обробку помилок».

Що вміє Cursor?

3

Пише код за описом

«Напиши функцію яка перемішує масив» – і готово. Розуміє контекст всього твого проєкту.



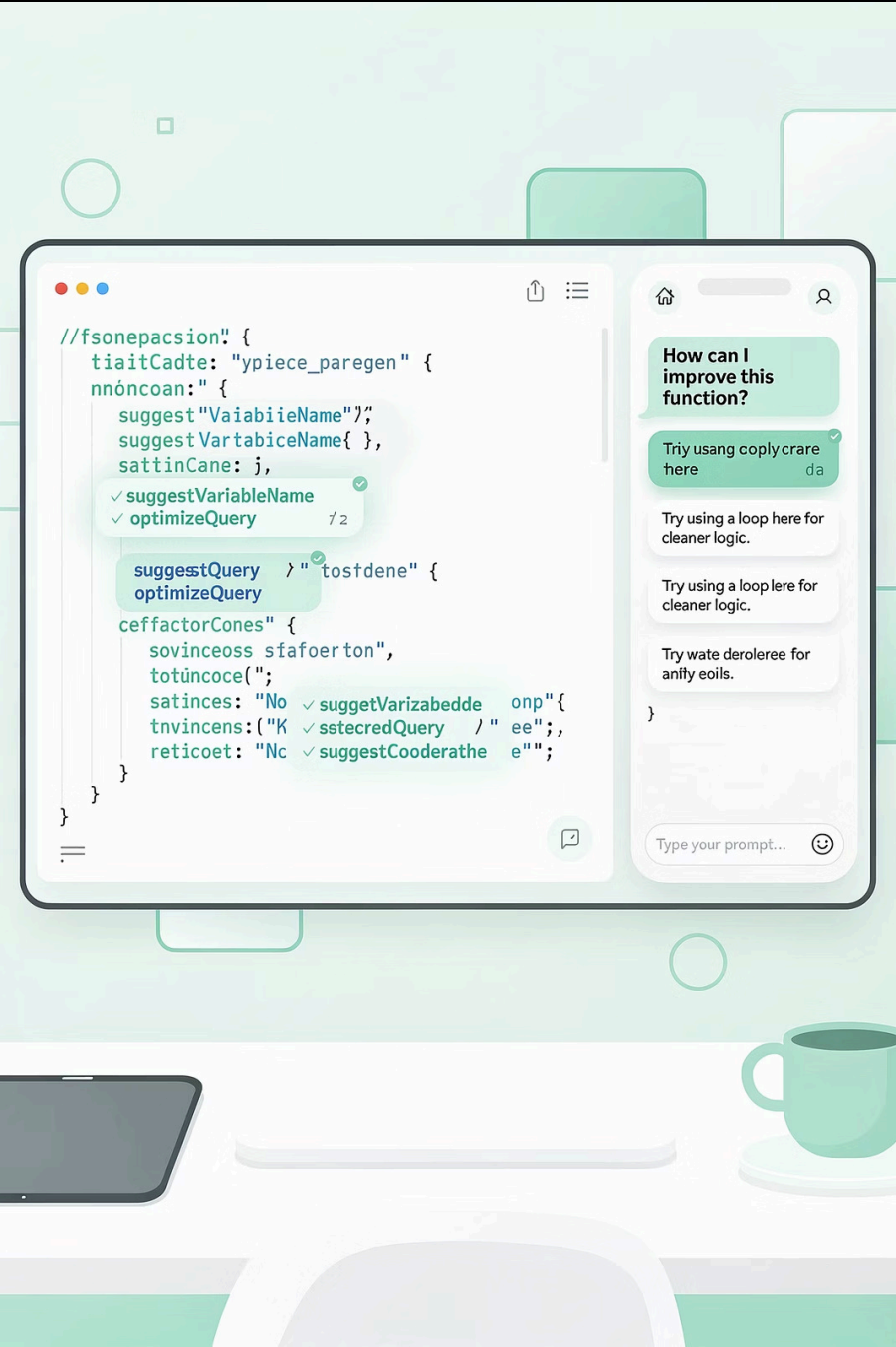
Знаходить помилки

Вставив код з помилкою? Cursor підсвічує і пропонує виправлення ще до запуску.



Пояснює чужий код

Скопіював код зі Stack Overflow і не розумієш? Виділи і запитай – отримаєш пояснення простою мовою.



GitHub Copilot — AI прямо у VS Code

GitHub Copilot – це AI-помічник який вбудовується у VS Code і підказує код прямо під час набору. Як автодоповнення, але розумне.

Як підключити?



Встанови розширення

У VS Code відкрий Extensions (Ctrl+Shift+X), знайди «GitHub Copilot» і встанови. Потрібен акаунт GitHub.



Безкоштовно для студентів

GitHub дає безкоштовний Copilot для школярів і студентів через GitHub Education. Реєструйся на education.github.com.



Просто починай писати

Copilot сам пропонує продовження коду сірим текстом. Натисни Tab щоб прийняти пропозицію.

Фішки Copilot



Автодоповнення цілих функцій

Написав назву функції і коментар – Copilot допише всю реалізацію. Часто з першого разу правильно.



Copilot Chat

Ctrl+I відкриває чат прямо в редакторі. Питати: «що робить цей код?», «напиши тести», «виправ помилку».



Розуміє твій проєкт

Copilot бачить всі відкриті файли і підлаштовує підказки під твої стиль коду і назви змінних.





ВАЙБ КОДІНГ

ChatGPT і Claude — AI для коду

ChatGPT і Claude — це чат-боти від OpenAI і Anthropic. Вони не вбудовані в редактор, але чудово пишуть, пояснюють і виправляють код у звичайному чаті.

ChatGPT (chat.openai.com)



Пиши код за описом

«Напиши Telegram бота на Node.js який надсилає випадковий жарт» – отримаєш готовий код з поясненням.



Виправляй помилки

Скопіюй помилку з терміналу і вставте в чат: «Чому ця помилка і як виправити?» – пояснить і покаже рішення.



GPT-4o — найкраща модель

Безкоштовна версія GPT-4o mini вже дуже потужна для навчання. Платна GPT-4o ще краще розуміє складний код.

Claude (claude.ai)



Великий контекст

Claude може прочитати весь твій файл з кодом одразу і відповісти з урахуванням всього. Ідеально для великих проєктів.



Краще пояснює

Claude відомий тим що пояснює код дуже зрозуміло і детально. Добре підходить для навчання.



Claude Sonnet безкоштовно

Безкоштовний план дає доступ до Claude Sonnet – дуже потужної моделі. Реєстрація без кредитки.

Як правильно просити AI писати код?

Погані запити vs хороші

«Зроби бота»

Занадто розмито. AI не знає що саме робити.

«Зроби Telegram бота на Node.js з Telegraf який відповідає на команду /joke випадковим жартом з масиву»

Конкретно! AI напише саме це.

«Виправи помилку»

Яку помилку? Де? AI не бачить твої екран.

«Ось помилка: [текст помилки]. Ось мій код: [код]. Що не так і як виправити?»

Ідеально. AI має всю інформацію.

Формула хорошого запиту



Що зробити?

Чітко опиши результат: «напиши функцію», «виправ помилку», «поясни цей код».



Яка технологія?

Вкажи мову і бібліотеки: «на JavaScript», «з Telegraf», «в Node.js».



Вставте контекст

Додай свій код або текст помилки. AI не бачить твої екран – йому потрібен текст.



Уточнюй і проси ще

Не вишло з першого разу? Напиши «не працює, ось помилка» і продовжуй діалог.

ВАЙБ КОДІНГ

Налаштування робочого місця для вайб кодингу

Що вже є 

Node.js – встановлено

VS Code – встановлено

Що потрібно додати

Cursor IDE

AI-редактор (альтернатива VS Code з вбудованим AI)

cursor.sh

GitHub Copilot

AI-розширення для VS Code

marketplace.visualstudio.com → шукай "GitHub Copilot"

Claude

AI-чат для коду у браузері

claude.ai

ChatGPT

AI-чат для коду у браузері

chatgpt.com

Обери свій стиль вайб кодингу

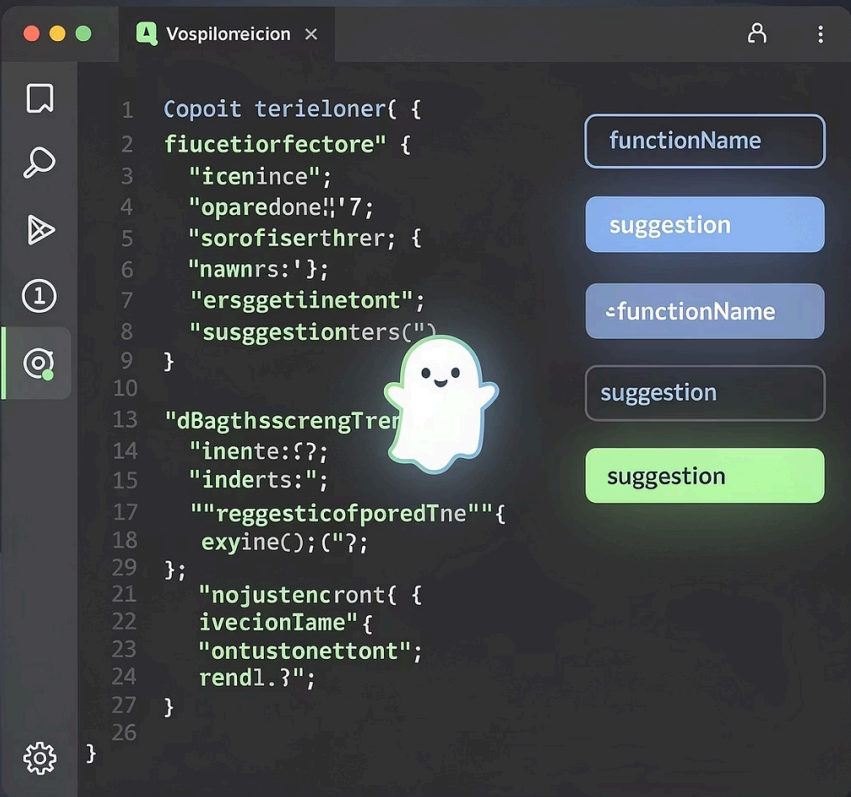
 VS Code + Copilot

 Cursor IDE

 Claude у браузері

ChatGPT у браузері





ВАЙБ КОДІНГ

Вайб кодинг у VS Code + GitHub Copilot

Встановлення Copilot



Відкрий VS Code



Натисни **Ctrl+Shift+X** (Extensions)



Знайди "GitHub Copilot" і натисни Install



Увійди через GitHub акаунт (безкоштовно для студентів!)



Посилання:
marketplace.visualstudio.com/items?itemName=GitHub.copilot

Як отримати безкоштовно?

Студенти та школярі можуть отримати GitHub Copilot безкоштовно через GitHub Education: education.github.com

Як використовувати?



Tab — прийняти підказку AI

(сіра підказка з'являється автоматично)



Ctrl+Enter — відкрити всі варіанти Copilot



Ctrl+I — Copilot Edits

Попроси AI змінити код прямо у файлі



Ctrl+Shift+I — Copilot Chat

Чат з AI у боковій панелі

Процес вайб кодингу

01

Напиши коментар що хочеш зробити:

// команда /weather яка повертає погоду

02

Copilot автоматично пропонує код — натисни **Tab**

03

Якщо не підходить — натисни **Ctrl+Enter** і обери кращий варіант

04

Для складніших задач — **Ctrl+I** і опиши завдання текстом

Вайб кодінг у Cursor IDE

Встановлення Cursor

- 
Перейди на cursor.sh і завантаж інсталятор
- 
Встанови як звичайну програму
- 
При першому запуску — увійди через Google або GitHub
- 
Обери безкоштовний план Hobby (500 запитів/місяць)
- 
Cursor автоматично імпортує налаштування з VS Code!

Чому Cursor — найкращий для вайб кодінгу?

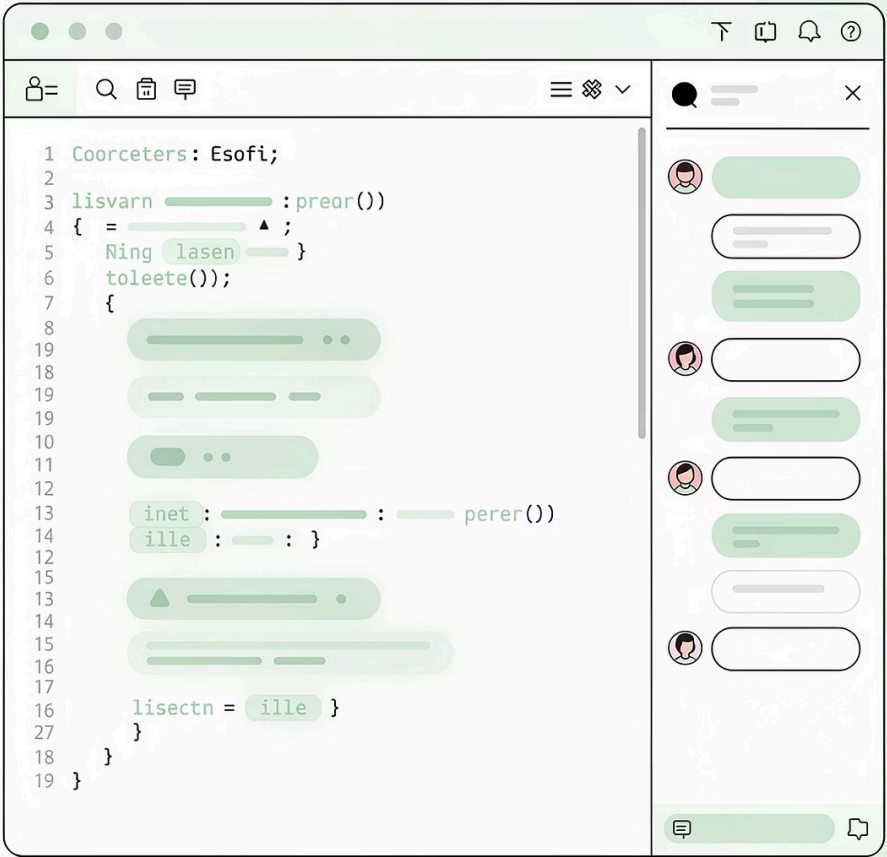
- AI** знає весь твій проєкт
бачить усі файли одночасно
- Composer:** AI пише код у кількох файлах одразу
- Вбудований чат**
не треба перемикатись між вкладками

Гарячі клавіші Cursor

- 
Ctrl+K
AI редагує виділений код або генерує новий
- 
Ctrl+L
відкрити AI чат (знає контекст проєкту)
- 
Ctrl+Shift+I
Composer: AI пише у кількох файлах
- 
Tab
прийняти автодоповнення
- 
@
прикріпити файл, папку або документацію до запиту

Процес вайб кодінгу

- Відкрий папку проєкту: File → Open Folder**
- Натисни Ctrl+L і опиши що хочеш зробити**
- AI бачить твій код і пропонує рішення з поясненням**
- Натисни "Apply" — код вставиться автоматично**
- Запусти: `node bot.js` і перевір результат**





ВАЙБ КОДІНГ

Cursor Agent — AI що сам пише весь проєкт

Що таке Cursor Agent?

Agent – це режим у Cursor, де AI діє самостійно: сам відкриває файли, пише код, виправляє помилки і запускає команди в терміналі. Ти просто описуєш завдання – Agent робить все сам.

Чим Agent відрізняється від звичайного чату?

Ctrl+L (чат): ти питаєш → AI відповідає → ти сам вставляєш код

Ctrl+Shift+I (Composer): AI пише у кількох файлах, але чекає твого підтвердження

Agent: AI сам виконує дії крок за кроком без твоєї участі

Як запустити Agent?



Натисни **Ctrl+Shift+I** (відкриє Composer)



У верхньому куті переключи режим з "Normal" на "Agent"



Опиши завдання: "Створи Telegram бота з командами /start, /help і /joke"



Натисни **Enter** і спостерігай як Agent сам все робить

Що вміє Agent?



Сам створює нові файли і папки



Сам встановлює npm пакети через термінал



Виправляє помилки автоматично після запуску



Редагує кілька файлів одночасно



Пояснює кожен крок у реальному часі

Безкоштовно vs Платно



Hobby (безкоштовно)

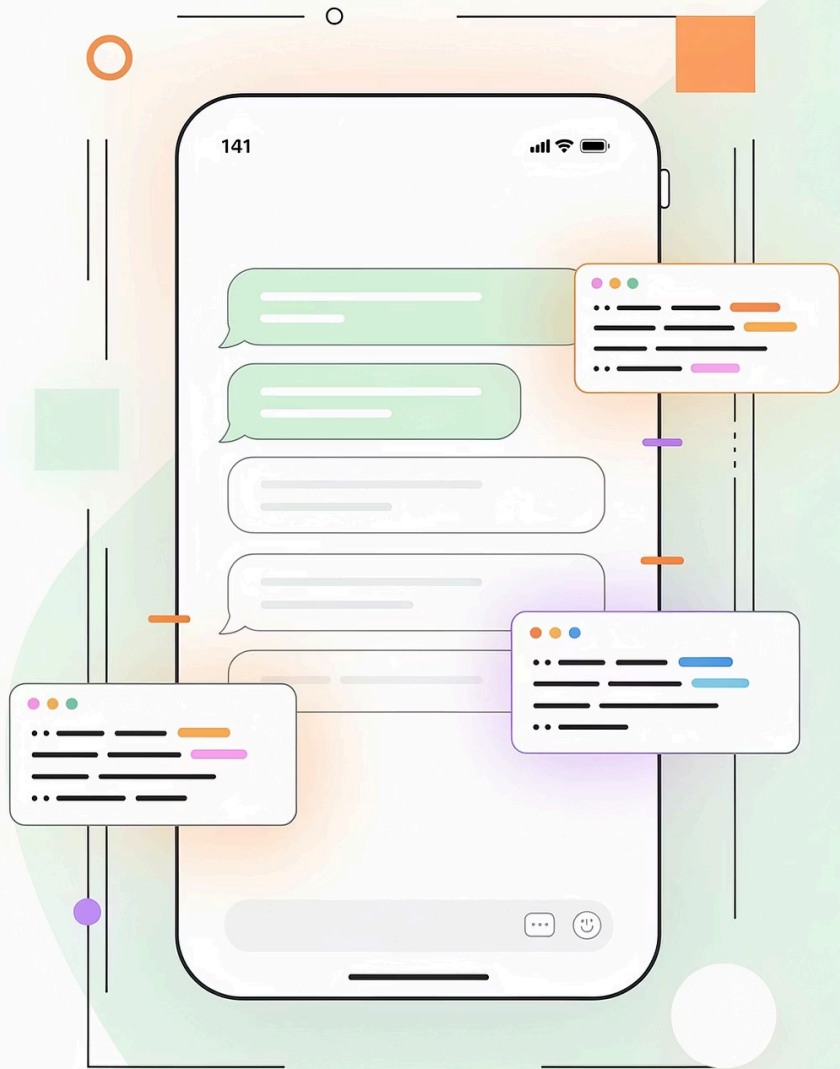
- 2 тижні пробного Pro
- Після: 50 запитів Agent/місяць
- 2000 автодоповнень/місяць
- Моделі: cursor-small, обмежено GPT-4o і Claude



Pro (\$20/місяць)

- 500 запитів Agent/місяць (швидкі)
- Необмежені повільні запити
- Необмежені автодоповнення
- Повний доступ: GPT-4o, Claude 3.5 Sonnet, o1

💡 Для навчання безкоштовного плану цілком достатньо – 50 запитів Agent на місяць це приблизно 1-2 повноцінних проєкти!



ВАЙБ КОДІНГ

Вайб кодінг з Claude (claude.ai)

Як почати?



Переїди на claude.ai



Зареєструйся безкоштовно (Google або email)



Безкоштовний план: ~20 запитів/день, Claude 3.5 Sonnet



Відкрий свій файл bot.js у VS Code



Копіюй код → вставляй у чат → отримуй відповідь → копіюй назад

Чому Claude підходить для коду?

Найкраще розуміє складний код і пояснює зрозуміло

Може тримати весь файл bot.js у контексті

Дає чіткі пояснення – ідеально для навчання

💡 Порада: у Claude можна завантажити файл bot.js напряму через скріпку – не треба копіювати вручну!

Як правильно вставляти код у Claude?



"Ось мій bot.js: [вставляєш код]. Додай команду /weather яка показує температуру в Києві через open-meteo API"



"Ось помилка: [вставляєш текст помилки]. Що не так і як виправити?"



Процес вайб кодінгу



Відкрий claude.ai у браузері поруч з VS Code



Опиши завдання + вставте поточний код



Claude поверне готовий код з поясненням



Скопіюй код у VS Code і запусти node bot.js



Якщо є помилка – вставте її в Claude і запитай як виправити

Вайб кодінг з ChatGPT (chatgpt.com)

Як почати?

- Переиди на chatgpt.com
- Зареєструйся безкоштовно (Google або email)
- Безкоштовний план: GPT-4o mini, необмежено
- GPT-4o: обмежена кількість запитів на день безкоштовно
- Відкрий VS Code поруч і починай вайб кодити!

Фішки ChatGPT для коду

Canvas режим: AI редагує код прямо у вікні, як у редакторі

Завантаження файлів: прикріпи `bot.js` напряму

Пам'ять між сесіями: ChatGPT пам'ятає твій проєкт (платна версія)

Як правильно просити ChatGPT?

“Я роблю Telegram бота на Node.js + Telegraf. Ось мій код: [код]. Додай команду /joke яка повертає випадковий жарт з масиву”

“Поясни цей код рядок за рядком: [вставляєш код]”

Процес вайб кодінгу

Відкрий chatgpt.com у браузері поруч з VS Code

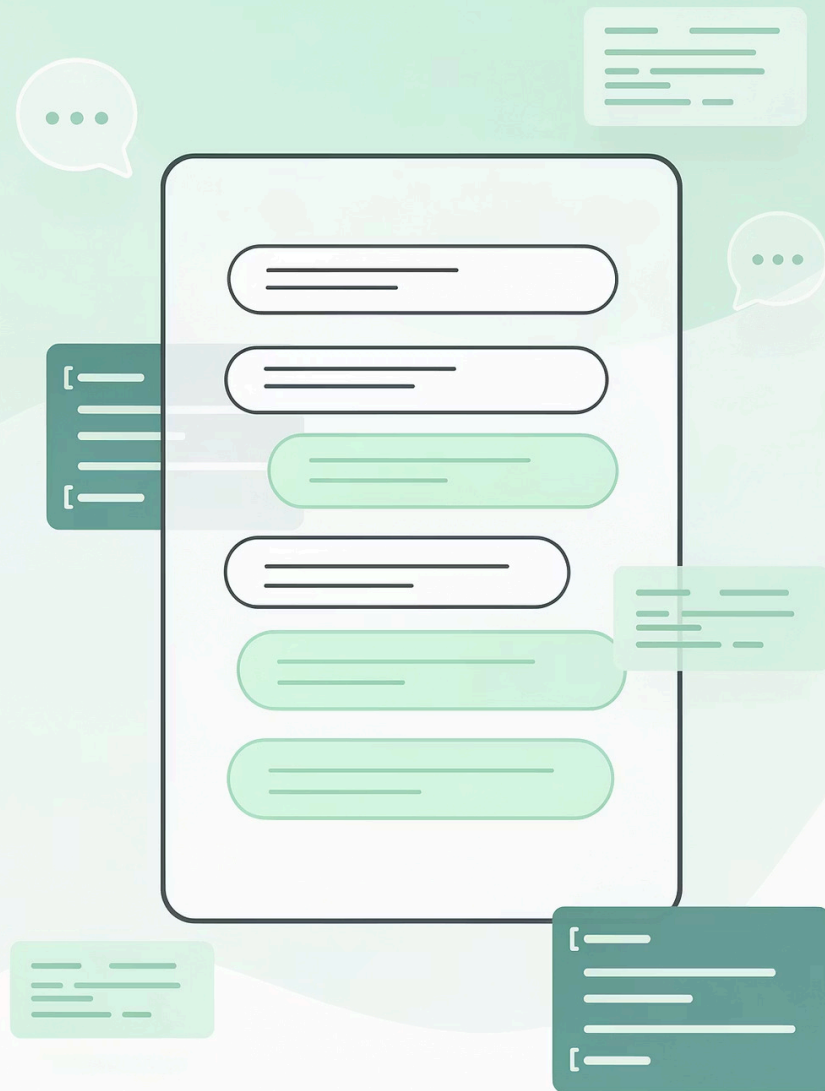
Опиши контекст: "Я роблю Telegram бота на Telegraf"

Вставте код і опишіть що хочеш додати або виправити

Скопіюй готовий код у VS Code

Запусти `node bot.js` і тести у Telegram

💡 Порада: використовуй ChatGPT Canvas (кнопка у правому куті чату) – AI редагуватиме код прямо у вікні без зайвого тексту!



ChatGPT Canvas — редагування коду як у редакторі

Що таке Canvas?

Canvas – це спеціальний режим ChatGPT, де AI відкриває окреме вікно з кодом поруч із чатом. Ти можеш редагувати код разом з AI, не копіюючи туди-сюди.

Звичайний чат: AI пише код у повідомленні → ти копіюєш вручну

Canvas режим: AI відкриває редактор → редагує код прямо там

Результат: менше копіювання, більше фокусу на коді

Як відкрити Canvas?

Відкрий chatgpt.com і почни новий чат

Напиши запит з кодом або натисни іконку Canvas (олівець) поруч з полем вводу

Або просто попроси: "Відкрий Canvas і напиши бота на Telegram"

Canvas відкриється автоматично справа

Що можна робити в Canvas?



Виділи частину коду → попроси AI змінити тільки її



Натисни на рядок → AI пояснить що він робить



Кнопка "Add comments" – AI додасть коментарі до всього коду



Кнопка "Fix bugs" – AI знайде і виправить помилки автоматично



Кнопка "Review code" – AI перевірить якість коду

Приклад роботи в Canvas



Вставляєш `bot.js` у Canvas



Виділяєш функцію `handleStart` і пишеш: "зроби відповідь більш дружньою"

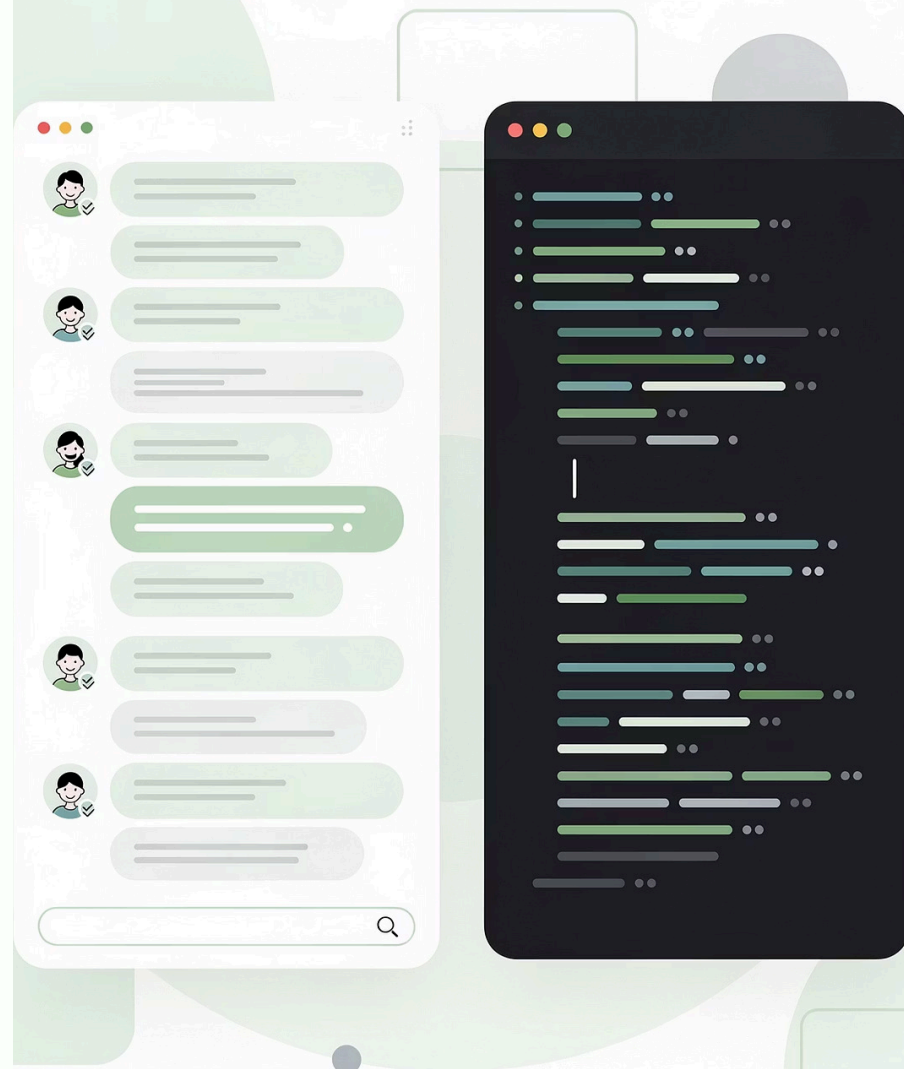


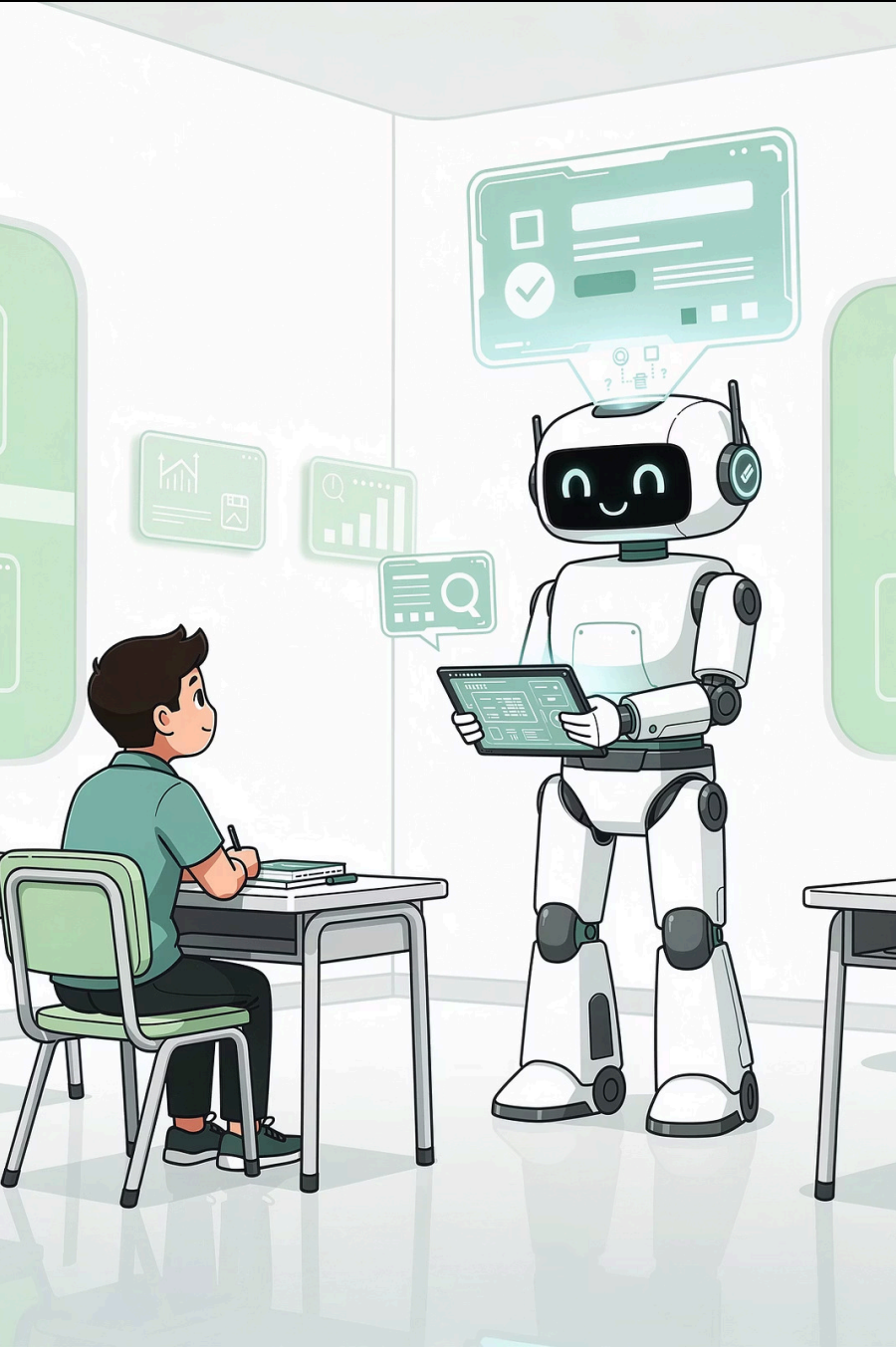
AI змінює тільки цю функцію, не чіпаючи решту



Натискаєш "Сору" і вставляєш у VS Code

💡 Canvas найкраще підходить для роботи з великими файлами – він бачить весь код одразу і не губить контекст!





ІДЕЯ БОТА

Бот-вікторина

Тестуй свої знання! Створи вікторини на улюблені теми: від історії до космічних кораблів. Бот ставитиме питання, а ти відповідатимеш, змагаючись з друзями або перевіряючи себе.



Різноманітні теми

Історія, математика, ігри,
що завгодно!



Зберігай результати

Відстежуй свій прогрес та
покращуй знання.

Бот-вікторина: деталі реалізації

Структура проєкту



questions.js — масив питань

Окремий файл із питаннями. Кожне питання – об'єкт із полями `text`, `options`, `correct`. Масив можна згенерувати через ChatGPT!



data.json — прогрес гравців

Зберігає рахунок кожного користувача між сесіями. Читається через `fs.readFileSync()`, оновлюється через `fs.writeFileSync()`.



sessions — поточний стан

Об'єкт `sessions` зберігає поточний рахунок і номер питання для кожного `userId` під час гри.



bot.js — головний файл

Підключає всі модулі, обробляє команди `/start`, `/quiz` та натискання `inline` кнопок.

Ключові моменти



Питання через ChatGPT

Попроси ChatGPT: «Згенеруй 10 питань для вікторини у форматі JS масиву з полями `text`, `options` (масив), `correct` (індекс)». Готово!



Сесія = поточна гра

`sessions[userId].score` – рахунок. `sessions[userId].questionIndex` – яке питання зараз. Скидається після завершення гри.



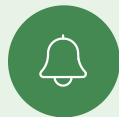
JSON = довгострокова пам'ять

`data.json` зберігає рекорди навіть після перезапуску бота. Ідеально для таблиці лідерів.

ІДЕЯ БОТА

Бот-нагадувач

Бот, який запам'ятовує твої нагадування і надсилає їх у потрібний час. Ідеально для дедлайнів, тренувань або важливих справ.



Встанови нагадування

Напиши боту що і коли нагадати – він збереже і надішле у потрібний момент.



Не пропусти важливе

Бот сам напише тобі у Telegram, навіть якщо ти забув відкрити застосунок.



Бот-нагадувач: деталі реалізації

Структура проєкту



reminders.json — список нагадувань

Кожне нагадування: { userId, text, time }. Зберігається у файлі, щоб не зникати після перезапуску бота.



Введення через сесію

sessions[userId] зберігає крок: спочатку бот питає «що нагадати?», потім «коли?». Відповіді зберігаються поетапно.



node-schedule для таймера

npm install node-schedule. schedule.scheduleJob(time, callback) – запускає функцію у потрібний час і надсилає повідомлення.



bot.telegram.sendMessage()

Надсилає повідомлення конкретному userId без очікування відповіді від користувача.

Ключові моменти



Формат часу

Використовує простий формат: «через 10 хвилин» або «HH:MM». Парсинг рядка через split(':') або бібліотеку ms.



Перезавантаження

При старті бота зчитує reminders.json і перепланує усі нагадування через scheduleJob – щоб вони не зникали після рестарту.



Видалення після спрацювання

Після надсилання нагадування – видали його з масиву і перезапиши файл. Інакше бот надсилатиме його знову і знову.

ІДЕЯ БОТА

Бот-словник

Бот перекладає слова, зберігає вивчені слова у список і допомагає повторювати їх у форматі флеш-карток.



Переклад на льоту

Напиши слово – бот одразу поверне переклад через API або зі свого словника.



Повторення слів

Команда /repeat показує випадкове слово зі збереженого списку для повторення.



Бот-словник: деталі реалізації

Структура проєкту



words.json — словник користувача

{ userId: [{ word: 'cat', translation: 'кіт' }, ...] }. Окремий список для кожного користувача.



LibreTranslate API — переклад

Безкоштовне API для перекладу. `axios.post()` з { q: word, source: 'en', target: 'uk' }. Можна замінити на власний словник у масиві.



sessions — режим додавання

`sessions[userId].mode = 'adding'` – коли користувач додає нове слово. Бот чекає на переклад у наступному повідомленні.



Math.random() для повторення

Команда `/repeat` бере випадкове слово зі списку і показує його як флеш-картку через `inline` кнопки «Знаю» / «Не знаю».

Ключові моменти



Додавання слів

Команда `/add` запускає сесію: бот питає слово, потім переклад. Зберігає пару у `words.json` під `userId`.



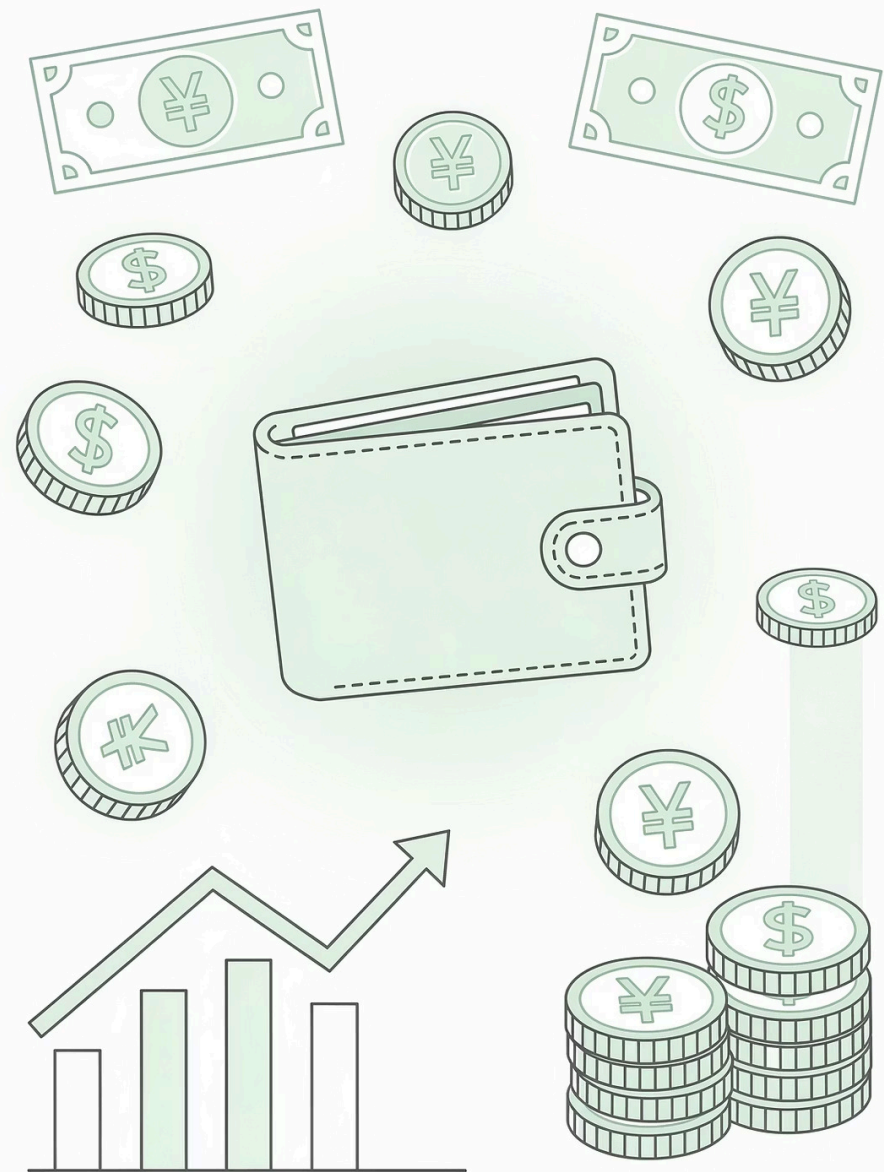
Перегляд списку

Команда `/list` виводить усі збережені слова. Якщо список великий – розби на сторінки по 10 слів через `slice()`.



Статистика

Зберігає поле `known: true/false` для кожного слова. Команда `/stats` показує скільки слів вивчено і скільки ще залишилось.



ІДЕЯ БОТА

Бот-трекер витрат

Бот допомагає вести облік особистих витрат: записує суму і категорію, показує статистику за день або місяць.



Додай витрату

Напиши суму і категорію – бот запише у файл з датою і userId.



Переглянь статистику

Команда /stats показує скільки витрачено за день, тиждень або по категоріях.

Бот-трекер витрат: деталі реалізації

Структура проєкту



expenses.json — всі витрати

{ userId: [{ amount: 150, category: 'їжа', date: '2024-04-07' }, ...] }. Кожен запис містить суму, категорію і дату.



Введення через сесію

sessions[userId].step: спочатку бот питає суму, потім пропонує обрати категорію через inline кнопки.



Категорії через кнопки

Markup.inlineKeyboard з категоріями: їжа, транспорт, розваги, інше. Обрана категорія зберігається через bot.action().



Фільтрація по даті

new Date().toISOString().slice(0,10) дає поточну дату у форматі YYYY-MM-DD. filter() відбирає витрати за потрібним днем.

Ключові моменти



Підрахунок суми

reduce() підсумовує витрати: expenses.reduce((sum, e) => sum + e.amount, 0). Можна рахувати по категоріях окремо.



Статистика за місяць

filter() по перших 7 символах дати (YYYY-MM) відбирає витрати за поточний місяць. Команда /month показує підсумок.



Скидання даних

Команда /clear видаляє всі витрати користувача зі списку і перезаписує файл. Корисно на початку нового місяця.

ІДЕЯ БОТА

Бот-рандомайзер для ігор

Бот допомагає вирішити суперечки в іграх: рандомно обирає команди, розподіляє ролі, кидає кубик або вибирає хто иде першим.



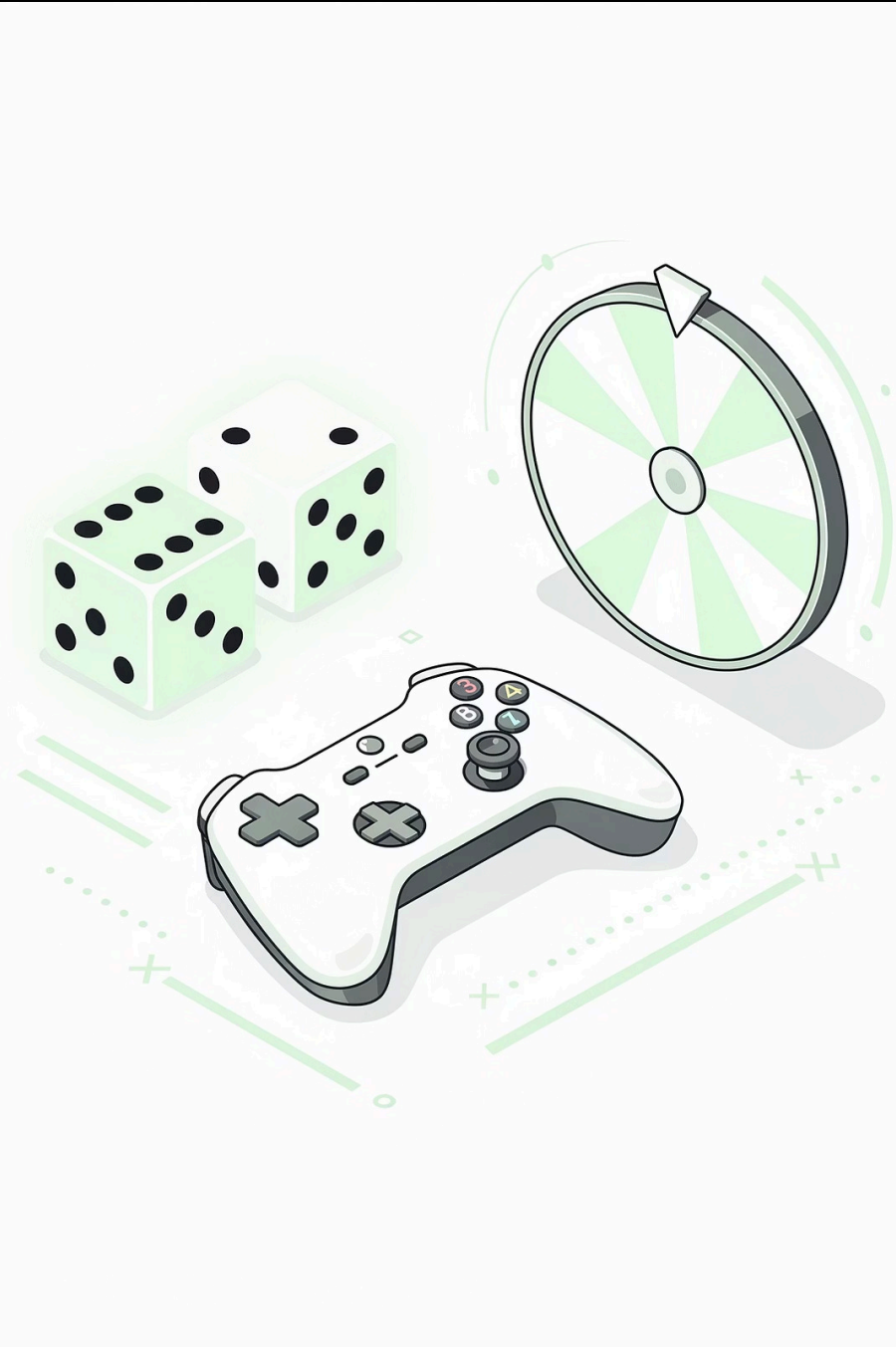
Рандом для всього

Хто иде першим? Які команди? Бот миттєво вирішує – без сварок і підозр у нечесності.



Для компанії

Введи список імен – бот розіб'є на команди, призначить ролі або обере випадкового переможця.



Бот-рандомайзер: деталі реалізації

Структура проєкту



players.js — масиви ролей і режимів

Окремии файл з масивами: roles (для Мафії, Among Us тощо), maps (карти в іграх), modes (режими гри). Легко доповнювати.



Введення імен через сесію

Команда /teams запускає сесію: бот просить вводити імена по одному. Команда /done завершує введення і ділить на команди.



Алгоритм розподілу

`sort(() => Math.random() - 0.5)` перемішує масив імен. Потім `slice()` ділить навпіл – перша половина команда 1, друга – команда 2.



Кубик і монетка

Команда /dice – `Math.floor(Math.random() * 6) + 1`. Команда /coin – `Math.random() > 0.5 ? 'Орел' : 'Решка'`.

Ключові моменти



Ролі для Among Us / Мафія

Масив ролей перемішується і роздається гравцям особисто через `bot.telegram.sendMessage(userId, role)` – кожен бачить тільки свою роль.



Реролл

Кнопка «Ще раз» перемішує і роздає ролі знову. Зберігаи список гравців у `sessions[chatId]` щоб не вводити повторно.



Групові чати

Бот може працювати в груповому чаті класу! `ctx.chat.id` замість `ctx.from.id` – і всі бачать результат рандому одразу.

ІДЕЯ БОТА

Бот-лічильник Robux

Бот допомагає відстежувати скільки Robux витрачено, скільки залишилось і на що копиш. Ніколи більше не витратиш заивого!



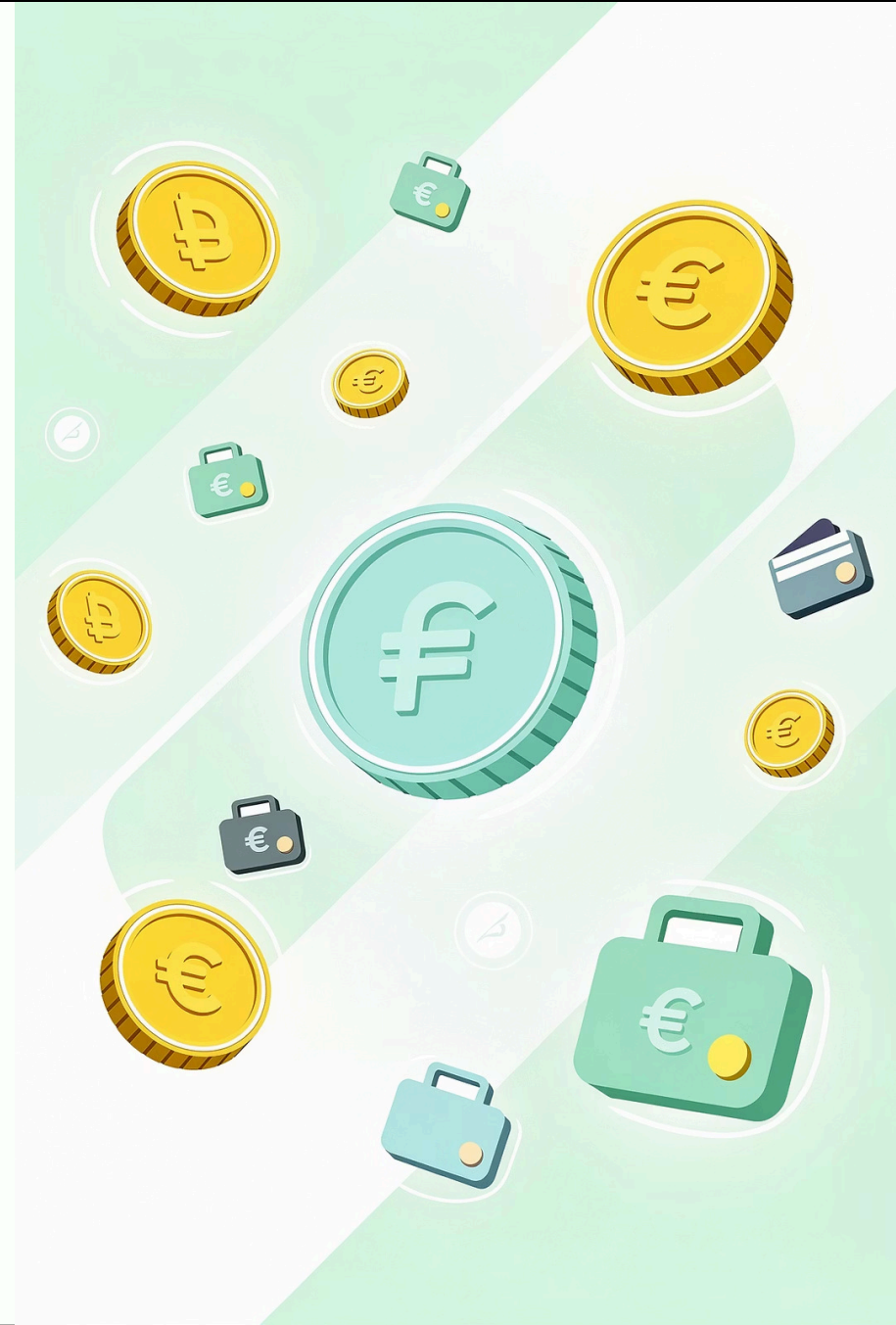
Записуй витрати

Написав /spent 400 – бот відніме від балансу і запише на що витратив. Завжди знаєш куди пішли Robux.



Копи на мрію

Встанови ціль – наприклад 1700 Robux на геимпас. Бот показує скільки ще залишилось накопичити.



Бот-лічильник Robux: деталі реалізації

Структура проекту



robux.json — дані користувача

{ userId: { balance: 1200, goal: 1700, goalName: 'геимпас Adopt Me', history: [...] } }. Окремии запис для кожного.



Поповнення балансу

Команда `/add 400` – `parseInt()` зчитує суму з тексту команди через `ctx.message.text.split(' ')[1]`. Додає до `balance` і зберігає.



Витрата з коментарем

Команда /spent 200 скін – бот відніме 200 і запише 'скін' у history з датою. Команда /history показує останні 5 витрат.



Ціль накопичення

Команда /goal 1700 геимпас встановлює ціль. Бот рахує відсоток прогресу і показує прогрес-бар з символів:

Ключові моменти



Прогрес-бар у тексті

Генерує прогрес-бар рядком: `const filled = Math.round(percent / 10);`
`'█'.repeat(filled) + '▒'.repeat(10 - filled)`. Виглядає круто прямо в чаті!



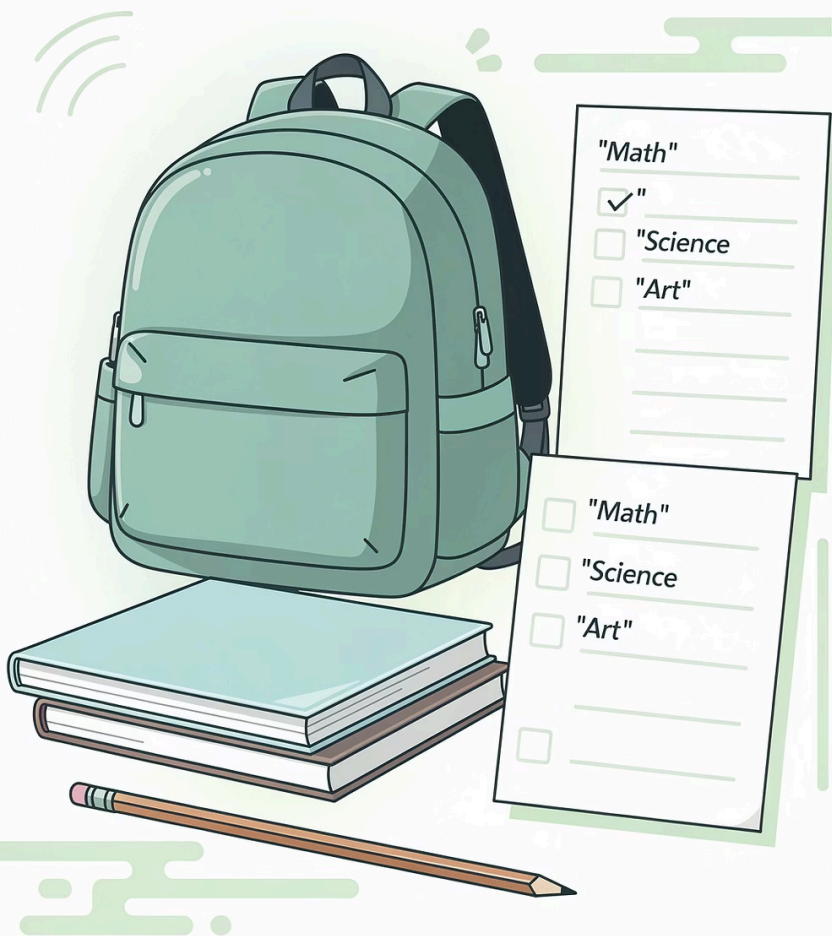
Мотиваційні повідомлення

Коли залишилось менше 10% – бот надсилає «Маиже! Ще трохи!». При досягненні цілі – «Вітаю! Можеш купувати!» з конфеті-емодзі.



Скидання після покупки

Команда /bought скидає баланс до нуля і очищає ціль. Бот запитує підтвердження через inline кнопки «Так» / «Ні» перед скиданням.



ІДЕЯ БОТА

Бот-домашка

Бот зберігає домашні завдання по предметах, нагадує про них і дозволяє відмічати виконані. Більше ніяких «я забув що задали»!



Записуй завдання

Команда /add математика
– прочитати параграф 5.
Бот збереже і нагадає
наступного ранку.



Відмічай виконане

Команда /done або кнопка
в списку – бот закреслює
завдання і показує скільки
ще залишилось зробити.

Бот-домашка: деталі реалізації

Структура проєкту



homework.json — список завдань

{ userId: [{ subject: 'математика', task: 'параграф 5', done: false, date: '2024-04-07' }, ...] }.



Додавання через сесію

Команда /add запускає сесію: бот питає предмет, потім завдання.
Або одразу: /add математика параграф 5 – парсинг через split(' ', 2).



Список із кнопками

Команда /list виводить завдання. Кожне – окреме повідомлення з inline кнопкою «Виконано». bot.action() відмічає done: true і оновлює файл.



Ранкове нагадування

node-schedule запускає щодня о 7:30 і надсилає список невиконаних завдань через bot.telegram.sendMessage(userId, text).

Ключові моменти



Фільтр по предмету

Команда /math показує тільки завдання з математики. filter(t => t.subject === 'математика') – і виводиш окремий список.



Спільний список класу

Один бот для всього класу в груповому чаті: завдання додає будь-хто, бачать усі. Зберігати по chatId замість userId.



Очищення виконаних

Команда /clear видаляє всі done: true завдання. Зручно робити щотижня. Бот підтверджує: «Видалити 5 виконаних завдань?».

ІДЕЯ БОТА

Бот-мем генератор

Бот надсилає випадковий мем з колекції, дозволяє додавати свої і влаштовувати голосування – який мем найсмішніший у класі.



Рандомний мем

Команда /meme – бот надсилає випадкову картинку з папки або масиву посилань. Кожен раз щось нове!



Голосування

Під кожним мемом кнопки «Смішно» і «Не смішно». Бот рахує голоси і показує рейтинг топ-мемів.

FUNNY MEME

LOL

SO TRUE



SO TRUE



Бот-мем генератор: деталі реалізації

Структура проєкту



memes.js — масив посилань

Масив URL-адрес картинок: `const memes = ['https://...', 'https://...', ...]`. Посилання можна зібрати вручн у або попросити ChatGPT згенерувати список.



Надсилання картинки

`ctx.replyWithPhoto(memes[randomIndex])` надсилає зображення. Одразу після – `ctx.reply` з inline кнопками «Смішно» і «Не смішно».



votes.json — рейтинг мемів

`{ 'url_мему': { funny: 12, notFunny: 3 } }`. Кожне натискання кнопки оновлює лічильник через `bot.action()`.



Топ мемів

Команда `/top` сортує `votes.json` за кількістю голосів 'funny' через `sort()` і виводить топ-3 з результатами голосування.

Ключові моменти



Додавання своїх мемів

Команда `/add` – бот просить надіслати картинку. `ctx.message.photo[0].file_id` зберігає Telegram `file_id` у `memes.json`. Не потрібен зовнішній хостинг!



Груповий чат класу

В груповому чаті всі бачать меми і голосують разом. Зберігає хто вже голосував (`voters: [userId]`) щоб не можна було голосувати двічі.



Щоденний мем

`node-schedule` надсилає випадковий мем щодня о 12:00 всім підписникам. Команда `/subscribe` додає `userId` до списку розсилки в `subscribers.json`.

ІДЕЯ БОТА

Бот-турнірна сітка

Бот організовує міні-турніри між друзями – в будь-яку гру, настільний теніс або FIFA. Автоматично будує сітку, оголошує пари і фіксує результати.



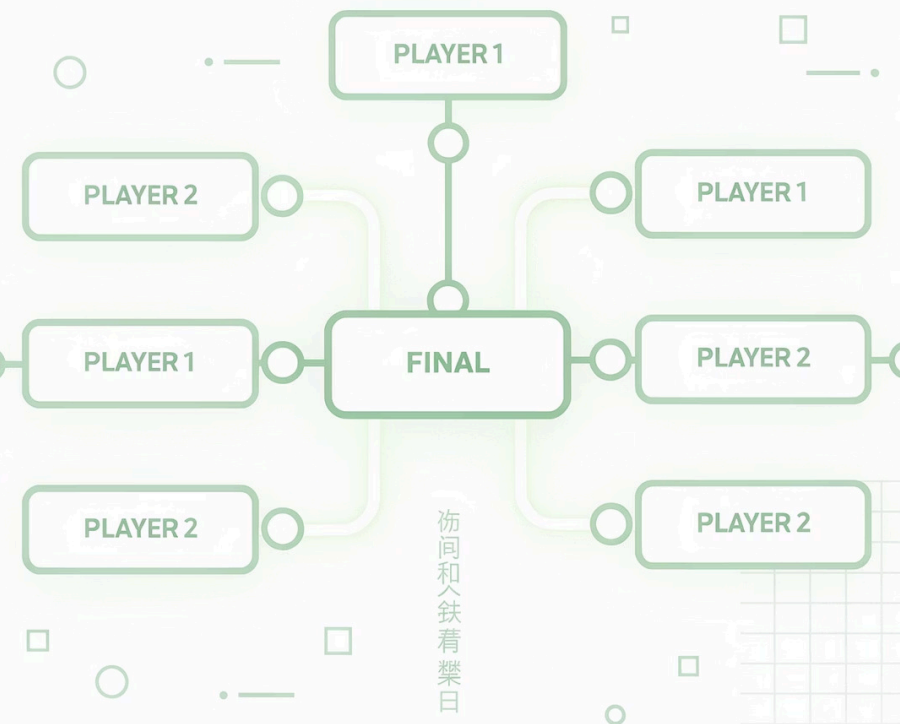
Автоматична сітка

Введи імена учасників – бот сам розіб'є на пари, перемішає і побудує турнірну сітку.



Фіксує результати

Команда /win Артем – бот записує перемогу, оновлює сітку і оголошує наступну пару.



Безкоштовні інструменти для вайб кодингу

Окрім вже згаданих Copilot, Claude та Cursor, існує безліч інших чудових, здебільшого безкоштовних ресурсів, які допоможуть вам у вайб кодингу:



Google Gemini

Безкоштовний мультимодальний ШІ, чудово підходить для генерації коду, пояснень та відповідей на технічні питання. Особливо ефективний для швидких запитів та досліджень.



Tabnine

AI-асистент для автодоповнення коду, інтегрується в багато IDE. Має щедрі безкоштовні тарифи для індивідуальних розробників, значно прискорюючи написання типових фрагментів коду.



Replit AI

ШІ, вбудований в онлайн-середовище розробки Replit. Ідеально підходить для швидкого прототипування, навчання та спільної роботи, надаючи допомогу в режимі реального часу.



Open-source моделі

Проекти на кшталт Code Llama, StarCoder (через Hugging Face) пропонують потужні ШІ-моделі, які можна запускати локально або використовувати через API з безкоштовними рівнями.

Стратегія використання "умовно безкоштовних" сервісів

Як працювати безперебійно та ефективно



Ротація сервісів

Як тільки вичерпався ліміт одного сервісу (наприклад, Copilot), перемикайтесь на інший (Claude, Gemini). Це дозволить вам працювати безперебійно.



Пріоритизація завдань

Для критично важливих або складних запитів використовуйте ШІ, який найкраще себе зарекомендував, навіть якщо він має менші ліміти. Прості питання можна адресувати іншим.



Локальні моделі

Якщо у вас є достатньо потужне обладнання, розгорніть open-source моделі локально. Це забезпечить необмежене використання і конфіденційність, але потребує початкового налаштування.



Комбінуйте інструменти

Використовуйте кожен інструмент для його сильних сторін – Copilot для швидкого автодоповнення, Claude/Gemini для генерації цілих блоків коду та рефакторингу, а Cursor для інтегрованого досвіду.

Google Gemini — безкоштовний ШІ для коду

Частина 1: Як почати Покроковий посібник

01

Доступ до Gemini

Зайдіть на gemini.google.com, увійдіть за допомогою облікового запису Google. Це ваш робочий простір для виб кодингу.

03

Пояснення коду

Вставте складний фрагмент коду та попросіть: "Поясни цей код рядок за рядком, що робить кожна частина і навіщо вона потрібна."

05

Фіча для Telegram бота

Попросіть: "Напиши функцію Telegraf.js, яка відповідає на команду /start привітанням і додає inline кнопку 'Детальніше'."

02

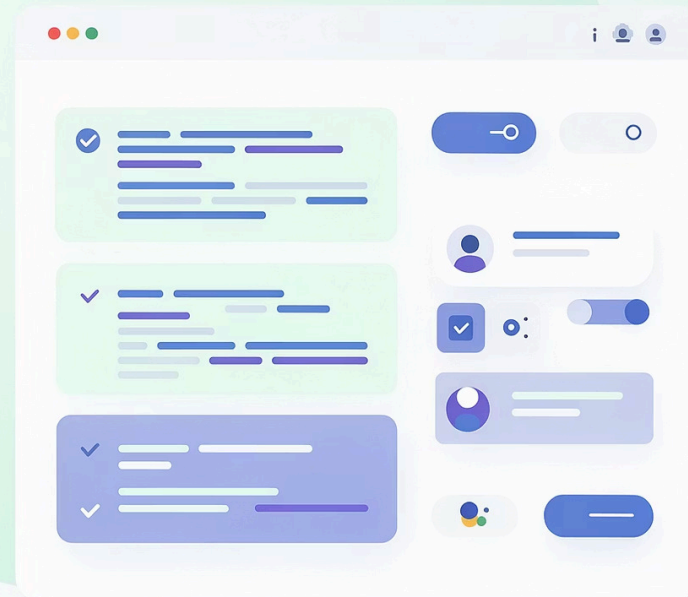
Генерація коду

Сформулюйте чіткий запит, наприклад: "Напиши функцію JavaScript для валідації email-адреси, яка повертає true або false."

04

Дебагінг та виправлення

Вставте код, що містить помилку, разом з повідомленням про помилку і попросіть: "Знайди помилку в цьому коді та запропонуй виправлення."



Google Gemini — практичні поради

Частина 2: Як отримати максимум

Оптимальне використання

Gemini ідеально підходить для швидкої генерації невеликих функцій, рефакторингу, пояснення складних концепцій та написання boilerplate-коду.

Безкоштовні ліміти

Сервіс надає щедрі безкоштовні ліміти, яких зазвичай достатньо для повсякденного навчання та невеликих проєктів. Використовуйте розумно.

Інтеграція з Telegraf/Node.js

Чудово генерує синтаксис та структури Telegraf, допомагає в роботі з JSON файлами, обробці промісів та асинхронних операцій у Node.js.

Приклади ефективних промптів

два приклади: "Напиши обробник для Telegraf бота, який записує user ID та ім'я користувача у файл users.json при першому запуску." та "Згенеруй фрагмент коду на Node.js для відправки HTTP POST запиту до API з JSON-тілом."

РЕСУРСИ

Tabnine — AI автодоповнення у вашій IDE

Частина 1: Встановлення та початок роботи

01

Встановлення

Знайдіть "Tabnine" у Marketplace розширень VS Code або вашої IDE та встановіть його. Це займе лише кілька секунд.

02

Активация безкоштовного плану

Після встановлення Tabnine запропонує увійти. Зареєструйтесь, щоб активувати щедрі безкоштовні тарифи для індивідуального використання.

03

Робота автодоповнення

Tabnine надає контекстуальні підказки, які з'являються безпосередньо у вашому редакторі під час набору коду.

04

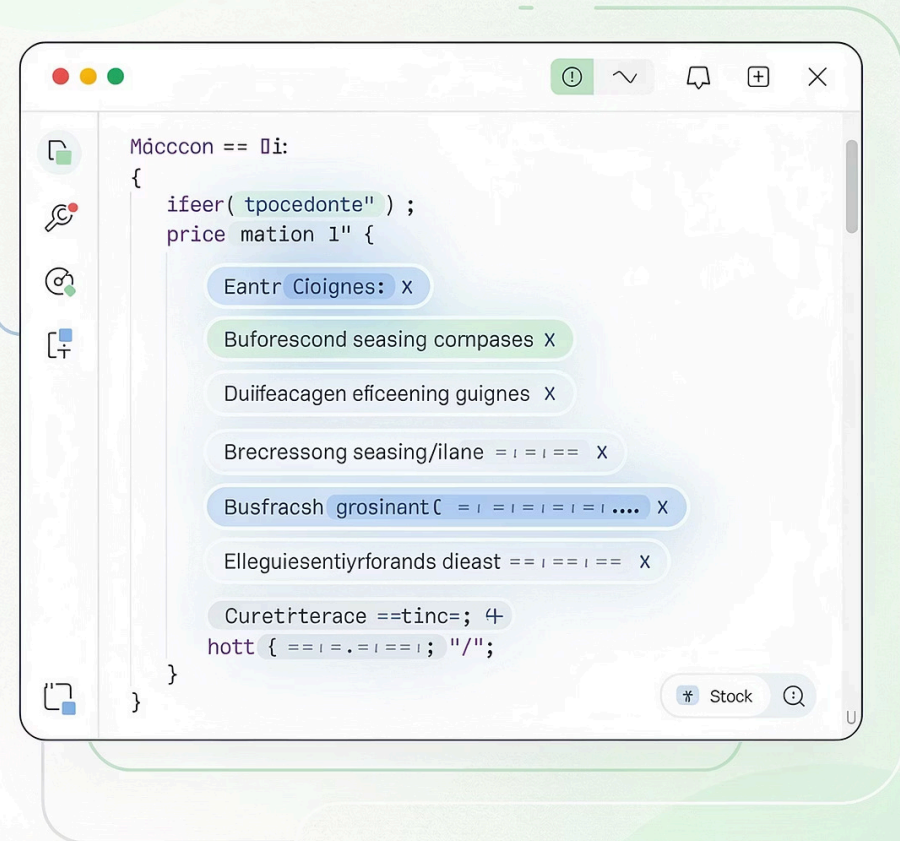
Прийняття та відхилення

Натисніть `Tab` для прийняття пропозиції або `Esc`, щоб її відхилити. Простір для курсора залишається вільним.

05

Ефективне кодування ботів

Дозвольте Tabnine прискорити написання типових обробників, функцій API та структур даних, таких як JSON для конфігурації бота.



Tabnine — практичні поради

Частина 2: Як отримати максимум

Переваги Tabnine

Ідеально підходить для швидкого, локального автодоповнення коду, фокусуючись на конфіденційності та продуктивності розробника.

Порівняння з Copilot

Tabnine менш агресивний, надає більш точні "локальні" підказки, базуючись на вашому коді, а не на глобальних даних, що робить його швидшим для певних завдань.

Покращення пропозицій

Пишіть чистий, послідовний код з осмисленими іменами змінних. Це допоможе Tabnine надавати більш релевантні підказки.

Для Telegraf/Node.js

Особливо корисний для швидкого автодоповнення методів Telegraf, обробки промісів, роботи з файловими операціями та структурою JSON об'єктів.

Replit AI — кодуй прямо у браузері

Частина 1: Реєстрація та перший проєкт

01

Реєстрація на Replit

Зайдіть на replit.com та створіть безкоштовний обліковий запис. Це ваш онлайн-інструмент для кодування.

03

Генерація коду з AI

Використовуйте вбудований AI Chat (бічна панель) для генерації функцій, класів або цілих скриптів. Сформулюйте чіткий запит.

05

Виправлення помилок

Якщо виникла помилка, скопіюйте повідомлення про неї в AI Chat та попросіть Replit AI знайти та запропонувати виправлення. Це значно прискорить дебагінг.

02

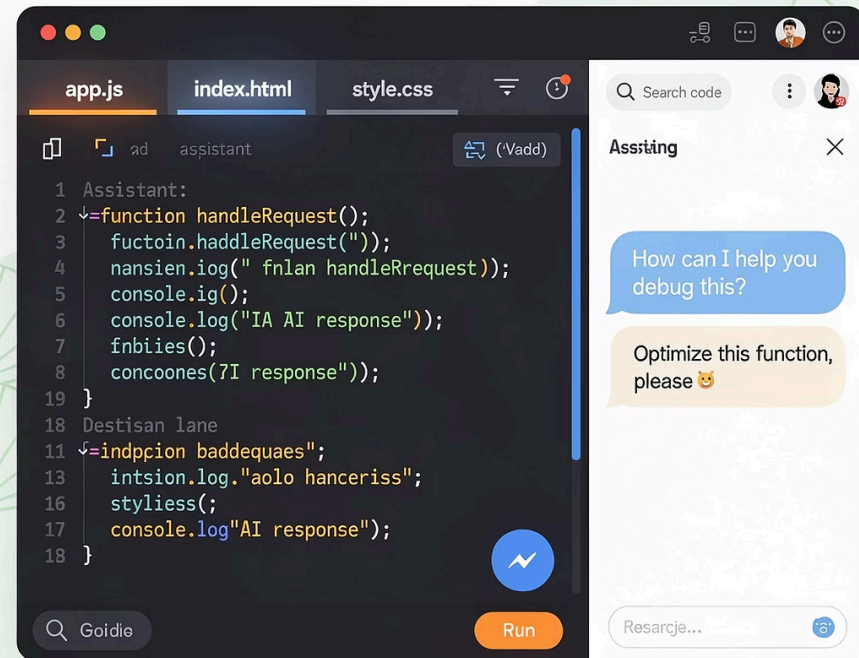
Створення проєкту

Натисніть "Create Repl", виберіть шаблон "Node.js" та назвіть свій проєкт. Ваш робочий простір буде готовий за лічені секунди.

04

Запуск бота

Після написання коду натисніть кнопку "Run". Replit автоматично запустить ваш Node.js додаток, наприклад, Telegram бота.



Replit AI — практичні поради

Частина 2: Як отримати максимум

Переваги для новачків

Replit усуває необхідність у локальному налаштуванні середовища розробки, дозволяючи зосередитися виключно на кодуванні та вивченні мови.

Встановлення npm пакетів

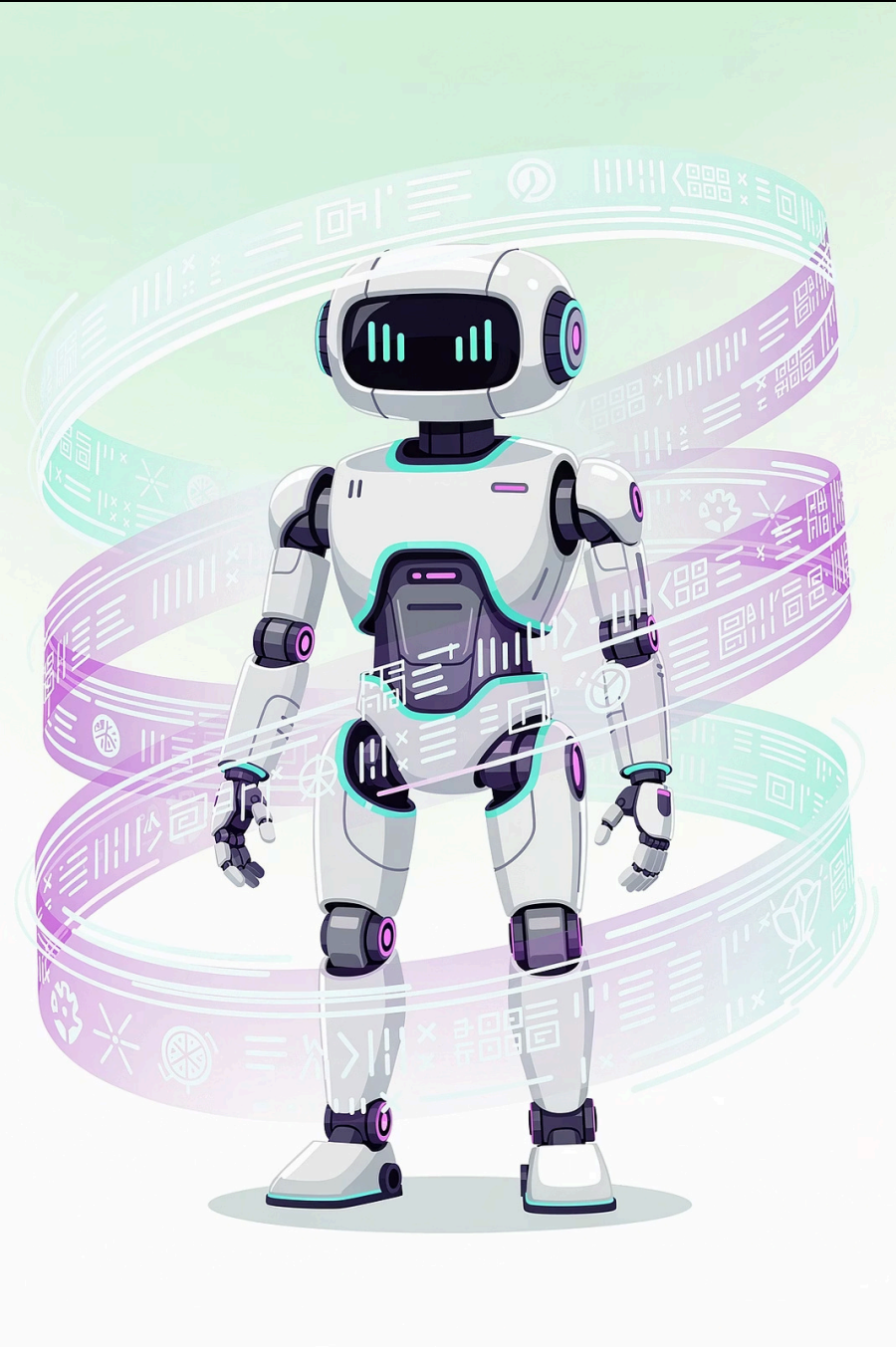
Replit автоматично встановлює залежності, зазначені у файлі `package.json`. Просто додайте потрібний пакет і збережіть файл, або використовуйте команду `npm install` у консолі.

Запуск Telegram бота

Щоб ваш бот працював безперервно, використовуйте команду `npm start` або `node index.js` у файлі `.replit` або в консолі. Replit підтримує постійний запуск.

Ліміти безкоштовного плану

Безкоштовний план Replit надає щедрі ресурси для навчання та невеликих проєктів, включаючи доступ до AI, обчислювальні години та дисковий простір. Для інтенсивних завдань можуть знадобитися платні тарифи.



РЕСУРСИ

Open-source моделі — необмежений АІ локально

Частина 1: Hugging Face та Ollama

01

Доступ до Hugging Face

Зайдіть на huggingface.co та знайдіть бажані моделі, такі як Code Llama або StarCoder.

02

Використання Inference API

Спробуйте більшість моделей онлайн безкоштовно через вбудований Inference API прямо на сторінці моделі.

03

Тестування в браузері

Багато моделей мають інтерактивні інтерфейси для тестування їхніх можливостей безпосередньо у вашому браузері.

04

Локальний запуск з Ollama

Встановіть [Ollama](https://ollama.com) для легкого запуску відкритих моделей на вашому комп'ютері.

05

Приклад: Ollama та бот-код

Запустіть `ollama run codellama` та попросіть: "Напиши функцію Python для Telegram бота, яка відповідає на команду /help з повідомленням про доступні команди."

Open-source моделі — практичні поради

Частина 2: Як отримати максимум

Переваги

Відкриті моделі забезпечують необмежене використання, повну конфіденційність даних та не вимагають API ключів для локального запуску.

Вимоги до обладнання

Для ефективного локального запуску моделей потрібна потужна відеокарта (GPU) з достатньою кількістю відеопам'яті (VRAM), особливо для великих моделей.

Code Llama vs StarCoder

Code Llama добре оптимізована для коду та математики, тоді як StarCoder має ширшу базу даних для загальніших завдань кодування.

Поради щодо промптів

Для досягнення найкращих результатів використовуйте чіткі, конкретні запити, завжди вказуйте мову програмування та бажану структуру виводу.

НОВИЙ МОДУЛЬ

Git та GitHub

Як зберігати, трекати та ділитись своїм кодом

git

Що таке Git



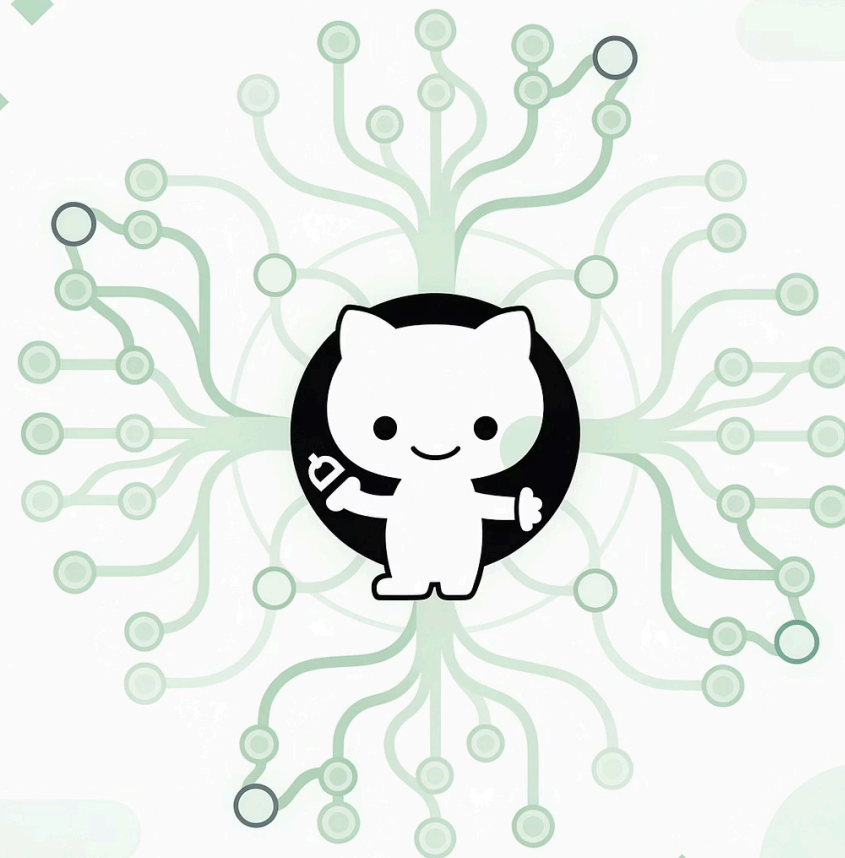
Перший репозиторій



Основні команди



Ділимося кодом



Що таке Git і навіщо він потрібен?

Уяви, що ти граєш у відеогру. Кожен раз, коли ти досягаєш важливої точки або просто хочеш зберегти свій прогрес, ти робиш «збереження». Якщо ти потім зробиш щось не так або гра зависне, ти завжди можеш повернутися до останнього «збереження». Без Git це все одно, що грати без збережень: якщо щось зламав, шляху назад немає. З Git ти завжди можеш «відкотитись» до будь-якого попереднього моменту свого коду.

Трекає зміни

Git запам'ятовує кожну зміну у твоєму коді: що додав, що видалив, коли і чому. Це повна історія твого проєкту.

Дозволяє повертатись

Зламав код? Не страшно. Git дозволяє повернутись до будь-якої попередньої, робочої версії за лічені секунди, врятувавши тебе від головного болю.

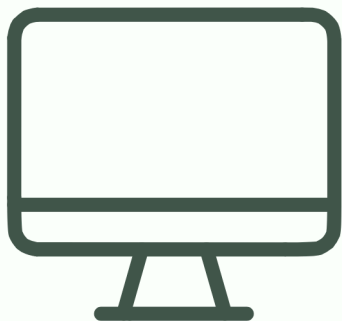
Командна робота

Кілька людей можуть працювати над одним проєктом одночасно, не заважаючи одне одному. Git допомагає об'єднувати їхні зміни без конфліктів.



Git vs GitHub — в чому різниця?

Git



Git – це програма, яка встановлюється на твої комп'ютер. Вона трекає зміни у файлах локально. Працює без інтернету. Це інструмент, як молоток – він є у тебе в руках.

- Встановлюється на ПК
- Працює офлайн
- Зберігає історію локально

GitHub



GitHub – це сайт в інтернеті, куди ти завантажуєш свій код. Це як Google Drive, але для коду. Дозволяє ділитись кодом з іншими.

- Сайт в інтернеті
- Зберігає код у хмарі
- Можна ділитись з іншими

Аналогія: Git – це твої щоденник, GitHub – це полиця в бібліотеці, де всі можуть його прочитати.

Встановлення Git

01

Зайди на git-scm.com

Відкрий браузер і переиди на офіційний сайт git-scm.com. Натисни велику кнопку "Download for Windows" (або macOS).

03

Перевір встановлення

Відкрий термінал (cmd або PowerShell) і введи: `git --version`. Якщо побачив щось на кшталт "git version 2.x.x" – все добре!

02

Запусти інсталятор

Відкрий завантажений файл. Натискай "Next" на всіх кроках – налаштування за замовчуванням підходять для початківців.

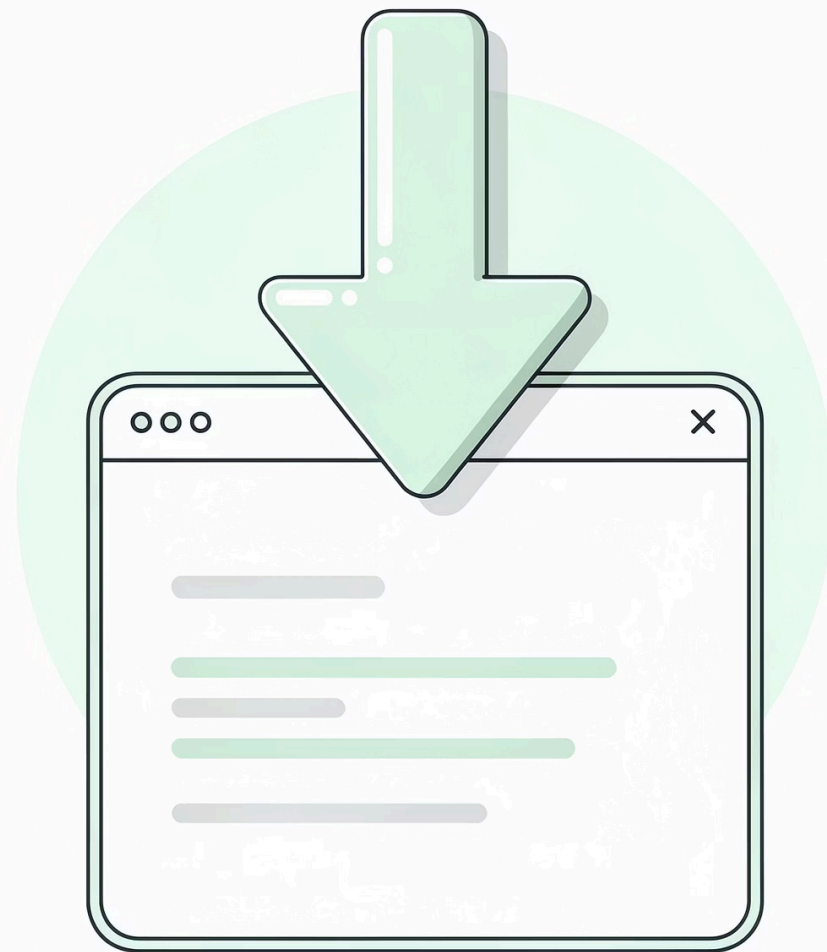
04

Налаштуй своє ім'я

Введи дві команди:

```
git config --global user.name "Твоє  
Ім'я"  
git config --global user.email  
"твій@email.com"
```

Це потрібно, щоб Git знав, хто робить зміни.





Create your account

DFF7E . tevilopan yor sloiare Password.

Username

Email address

Sign up

Shell pthy re steabe netm [Sepawordy](#)

☐ Drappy 2 decsy diounuaniy

GIT TA GITHUB

Реєстрація на GitHub

01

Зайди на github.com

Відкрий github.com у браузері. Натисни "Sign up" у правому верхньому куті.

02

Введи дані

Вкажи email, придумай пароль та унікальний username (нікнейм). Це твої публічний профіль – обери щось нормальне, бо роботодавці теж дивляться на GitHub!

03

Підтверди email

GitHub надішле листа на твою пошту. Відкрий його і натисни кнопку підтвердження.

04

Заповни профіль (опційно)

Додай аватарку та коротке біо. Це твоє "портфоліо" як розробника.

05

Готово!

Ти на GitHub! Тепер можна створювати репозиторії та ділитись кодом з усім світом.

Ключові поняття Git



Репозиторій (repo)

Це папка з твоїм проєктом, яку відстежує Git. Думай про неї як про "проєкт" – усі файли коду всередині.



Коміт (commit)

Це "збереження" у грі. Ти робиш коміт, коли хочеш зафіксувати поточний стан коду. Кожен коміт має повідомлення – що саме ти змінив.



Гілка (branch)

Це окрема "лінія розробки". Головна гілка називається main. Можна створювати нові гілки, щоб тестувати ідеї, не ламаючи основний код.



Push / Pull

Push – завантажити свій код на GitHub. Pull – завантажити чужі зміни до себе.

Перший репозиторій на GitHub

01

Натисни «New repository»

На головній сторінці GitHub натисни зелену кнопку "New" або "+" у верхньому меню → "New repository".

03

Обери налаштування

Visibility: Public (всі бачать) або Private (тільки ти). Постав галочку "Add a README file" – це опис твого проєкту.

05

Скопіюй URL репозиторію

Натисни зелену кнопку "Code" і скопіюй HTTPS посилання. Воно знадобиться для підключення.

02

Назви репозиторій

Введи назву, наприклад: my-telegram-bot. Назва не повинна мати пробілів – використовуй дефіс або підкреслення.

04

Натисни «Create repository»

Репозиторій створено! Ти побачиш сторінку з твоїм порожнім проєктом і інструкціями.

New repository



Repository name



Create new repository

Visibility Visibility



Public



Private

☐ Additionally options n☐ Private options☐ Additional options

Основні команди Git

Команди для початку роботи

```
git init
```

Ініціалізує Git у папці проєкту. Робиш один раз на початку.

```
git clone [URL]
```

Копіює репозиторій з GitHub на твої комп'ютер.

```
git status
```

Показує, які файли змінились і що ще не збережено.

Команди для збереження змін

```
git add .
```

Додає всі змінені файли до "черги" для коміту. Крапка означає "всі файли".

```
git commit -m "опис"
```

Зберігає зміни з повідомленням. Наприклад: `git commit -m "Додав команду /start"`

```
git push
```

Завантажує твої коміти на GitHub.